디지털 집적회로 — 수업 정리 노트

삼성 **DS System Expert Course** | 서울대 김재준 교수

Ch1. Digital Circuit Introduction

왜 이 강의를 듣는가?

이 강의의 핵심 메시지는 이것입니다. “훌륭한 시스템 설계자는 회로 부품이 어떻게 동작하는지, 소자의 특성이 무엇인지까지 이해해야 한다. 놀랍게도 그런 설계자는 매우 드물다.”

비유하자면, 최고의 요리사는 레시피만 아는 게 아니라 재료 자체를 깊이 이해합니다. 밀가루의 글루텐이 왜 반죽을 탄탄하게 만드는지, 고기가 왜 마이야르 반응을 일으키는지 알기 때문에 더 좋은 요리를 만들 수 있습니다. 마찬가지로, 칩을 설계하는 사람이 트랜지스터 하나가 어떻게 동작하는지 이해하면 훨씬 더 좋은 시스템을 만들 수 있습니다.

소프트웨어 개발자 관점에서 이렇게 생각해보면 됩니다. 여러분이 짠 코드가 결국 0과 1의 신호로 바뀌어서 실리콘 칩 위의 트랜지스터들을 켜고 끕니다. 그 트랜지스터가 어떻게 생겼는지, 왜 느려지는지, 왜 전력을 소모하는지 알면, 하드웨어를 더 잘 활용하는 소프트웨어를 만들 수 있습니다.

이 강의에서 배울 내용의 흐름:

```
Introduction → MOSFET → CMOS 회로 → Logical Effort
→ Wire → Clocking → Dynamic Circuit → Memory → SRAM
```

트랜지스터(Transistor)란 무엇인가?

우리가 쓰는 스마트폰에는 수백억 개의 트랜지스터가 들어있습니다. 이 모든 것의 기본 단위가 트랜지스터입니다.

이름의 유래: “Trans(변환)” + “Resistor(저항)”의 합성어입니다. 벨 연구소의 J. Pierce가 만든 이름입니다. 즉, **저항값을 외부에서 제어할 수 있는 소자**라는 뜻입니다.

트랜지스터의 종류는 여러 가지가 있습니다.

- MOSFET (Metal-Oxide-Semiconductor Field-Effect Transistor)
- BJT (Bipolar Junction Transistor)
- MESFET (Metal-Semiconductor FET)

이 중 현대 디지털 칩은 거의 전부 MOSFET을 사용합니다. 왜냐하면 MOSFET은 게이트에 전류가 거의 흐르지 않아 소비 전력이 낮고, 집적도를 높이기에 유리하기 때문입니다.

반도체(Semiconductor)란 무엇인가?

MOSFET을 이해하려면 먼저 반도체를 이해해야 합니다.

물질은 전기 전도도에 따라 세 종류입니다.

- **도체(Conductor)**: 전기가 잘 흐릅니다. 구리, 금 같은 금속.
- **절연체(Insulator)**: 전기가 안 흐릅니다. 유리, 고무.
- **반도체(Semiconductor)**: 조건에 따라 달라집니다. 대표적으로 실리콘(Si).

순수한 실리콘은 원자들이 공유결합으로 묶여 있어서 자유롭게 이동할 수 있는 전하가 없습니다. 전기가 거의 안 흐릅니다. 그런데 여기에 불순물을 아주 조금 섞어 넣으면(**도핑, Doping**) 전도성이 생깁니다.

N-type 반도체: 전자를 "기부"하는 도너(Donor)를 주입합니다. 자유롭게 이동하는 전자(음전하)가 많아집니다. 예: 인(P), 비소(As).

P-type 반도체: 전자를 "흡수"하는 억셉터(Acceptor)를 주입합니다. 전자가 빠져나간 빈자리인 정공(Hole, 양전하)이 많아집니다. 예: 붕소(B).

Shockley의 주차장 비유

노벨상을 받은 물리학자 쇼클리가 제안한 직관적인 비유입니다. 언덕에 있는 2층짜리 주차장을 상상하세요.

절연체 상태: 1층은 차가 꽉 차 있고, 2층은 텅 비어 있습니다. 1층은 공간이 없고 2층은 차가 없으니 아무것도 움직일 수 없습니다. 전류가 흐르지 않습니다.

진성 반도체(열 에너지로 일부 전자가 자유로워진 상태): 1층의 차 하나가 2층으로 올라갔습니다. 2층의 차는 자유롭게 움직일 수 있고(이것이 자유 전자), 1층에는 빈자리가 생겨서 옆 차들이 그 빈자리로 이동할 수 있습니다(이것이 정공의 이동). 두 방향으로 전류가 흐릅니다.

N-type (Donor 첨가): 외부에서 차를 2층에 "기증"해 줍니다. 2층에 여분의 차(자유 전자)가 생겨서 자유롭게 이동합니다. 전류가 흐릅니다.

P-type (Acceptor 첨가): 1층에서 차를 하나 "제거"합니다. 1층에 빈자리(정공)가 생겨서 옆 차들이 그 빈자리로 이동합니다. 정공이 이동하는 방향으로 전류가 흐릅니다.

MOSFET의 구조

MOSFET은 이름 그대로 금속(Metal) - 산화막(Oxide) - 반도체(Semiconductor)로 구성된 전계효과(Field Effect) 트랜지스터입니다.

```
  Gate(G) ← 제어 단자
      |
[금속판]
[SiO₂ 절연체] ← 전류 안 흐름. 전기장만 통과
[반도체]
N+      N+   ← Source(S)와 Drain(D)
└──P형 기판──┘
```

핵심은 Gate 아래의 산화막(SiO_2)이 절연체라는 겁니다. Gate에 전압을 걸어도 직접 전류가 흐르지 않습니다. 대신 전기장(Electric Field)이 산화막을 통과해서 아래 반도체 표면에 영향을 줍니다. 이것이 "전계효과(Field Effect)"입니다.

소프트웨어 비유로는 함수 호출과 비슷합니다. Gate는 "어떤 값을 전달"하는 게 아니라 "신호만 보내서" 트랜지스터를 켜고 끄는 것입니다.

NMOS 트랜지스터 동작

NMOS는 P형 기판 위에 N형 Source와 Drain이 만들어진 트랜지스터입니다. 캐리어는 전자(Electron)입니다.

OFF 상태 (Gate = 0V)

Gate에 전압이 없으면 전기장이 없습니다. Source와 Drain은 N형인데, 그 사이에 P형 기판이 있습니다. N-P-N 구조이므로 전류 통로가 없습니다. 스위치가 열린 상태입니다.

ON 상태 — 채널 형성 (Gate = 1V)

Gate에 양전압을 걸면 전기장이 기판 표면의 전자들을 끌어당깁니다. Gate 바로 아래에 N형 채널(Channel)이 형성됩니다. Source와 Drain 사이에 전자가 이동할 수 있는 통로가 생깁니다. 여기에 Drain에도 전압을 걸면 전자가 Source → Drain으로 이동하고, 전류는 반대 방향(Drain → Source)으로 흐릅니다.

NMOS 요약: Gate HIGH(1) → ON, Gate LOW(0) → OFF

PMOS 트랜지스터 동작

PMOS는 N형 기판 위에 P형 Source와 Drain이 만들어진 트랜지스터입니다. 캐리어는 정공(Hole)입니다. 모든 것이 NMOS와 반대입니다.

OFF 상태 (Gate = HIGH = VDD)

Gate가 HIGH이면 전기장이 기판 표면의 정공을 밀어냅니다. P형 채널이 생기지 않습니다. 차단 상태입니다.

ON 상태 (Gate = LOW = 0V)

Gate가 LOW가 되면 전기장이 반대가 되어 N형 기판 표면에 P형 채널이 형성됩니다. 정공이 Source → Drain으로 이동하고 전류도 Source → Drain 방향으로 흐릅니다.

PMOS 요약: Gate LOW(0) → ON, Gate HIGH(1) → OFF

NMOS와 PMOS가 정반대로 동작하는 이 특성이 CMOS(Complementary MOS)의 핵심입니다. “Complementary”가 “상보적”이라는 뜻입니다.

CMOS 인버터 — NOT 게이트 만들기

이제 실제로 논리 게이트를 만들어봅시다. “입력이 1이면 출력이 0, 입력이 0이면 출력이 1”인 NOT 게이트(인버터)입니다.

첫 번째 시도: NMOS만 사용

```
        출력 노드 (커패시터처럼 동작)
              |
   A ─┤NMOS├─┘
              |
             GND
```

A=1이면 NMOS가 켜져서 출력이 GND(0)로 연결됩니다. 출력 = 0. 좋습니다.

그런데 A=0이면 NMOS가 꺼집니다. 출력 노드는 어디에도 연결되지 않은 플로팅(Floating) 상태가 됩니다. 출력이 1이 되어야 하는데, 출력을 VDD까지 충전해줄 경로가 없습니다. 문제입니다.

해결책: PMOS를 위쪽에 추가

```
VDD
 |
[PMOS] ← A=0이면 ON → VDD에서 출력으로 전류
 |
 +─── 출력 Y
 |
[NMOS] ← A=1이면 ON → 출력에서 GND로 전류
 |
GND
```

A=0일 때: PMOS는 Gate=0이므로 ON, NMOS는 Gate=0이므로 OFF. VDD에서 출력으로 전류가 흘러 Y=1.

A=1일 때: PMOS는 Gate=1이므로 OFF, NMOS는 Gate=1이므로 ON. 출력에서 GND로 전류가 흘러 Y=0.

A	PMOS	NMOS	Y
0	ON	OFF	1
1	OFF	ON	0

완벽한 인버터입니다!

CMOS의 핵심 장점: 어떤 상태에서도 PMOS와 NMOS 중 **하나만** 켜집니다. 따라서 VDD에서 GND로 직접 흐르는 전류(단락, Short Circuit)가 없습니다. 정적 상태에서 전력 소비가 이론적으로 0입니다. 이것이 배터리로 동작하는 현대 모바일 기기의 핵심 기술입니다.

일반적인 CMOS 회로 구조

인버터에서 배운 구조를 일반화하면 모든 CMOS 논리 게이트에 적용됩니다.

```
           VDD
            |
   ┌──[PMOS Pull-up Network (PUN)]──┐
 입력들                               출력 Y
   └──[NMOS Pull-down Network (PDN)]┘
            |
           GND
```

- **Pull-up Network (PUN)**: PMOS들로 구성. 출력을 VDD(1)로 올리는 역할
- **Pull-down Network (PDN)**: NMOS들로 구성. 출력을 GND(0)로 내리는 역할

핵심 설계 규칙:

- PDN에서 NMOS를 **직렬**로 연결하면 AND 동작 (둘 다 켜져야 통함)
- PDN에서 NMOS를 **병렬**로 연결하면 OR 동작 (하나만 켜져도 통함)
- PUN은 PDN의 **반대** 구조 (PDN이 직렬이면 PUN은 병렬, 그 반대도 마찬가지)

CMOS NAND 게이트

NAND는 “A AND B의 반전”입니다. A=1이고 B=1일 때만 출력이 0, 나머지는 모두 1.

PDN에서 AND를 구현하려면 NMOS를 직렬로 연결합니다. A와 B 모두 1이어야 GND 경로가 만들어집니다.

```
        VDD
         |
    ┌─[PMOS A]─┐   ← A=0 또는 B=0이면 PUN이 출력을 VDD로 올림
    ├─[PMOS B]─┤   (병렬: 하나만 ON이어도 됨)
    |          |
    A, B       Y (출력)
               |
           [NMOS A]  ← A와 B 둘 다 1이어야 GND 통로 생김
               |    (직렬: 둘 다 ON이어야 함)
           [NMOS B]
               |
              GND
```

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

CMOS NOR 게이트

NOR는 “A OR B의 반전”입니다. 둘 다 0일 때만 출력이 1, 나머지는 모두 0.

PDN에서 OR를 구현하려면 NMOS를 병렬로 연결합니다. A 또는 B 중 하나만 1이어도 GND 경로가 생깁니다.

```
        VDD
         |
    [PMOS A]  ← 직렬: A=0 AND B=0이어야 PUN이 출력을 올림
         |
    [PMOS B]
         |
         Y (출력)
    ┌────┘
[NMOS A] [NMOS B]  ← 병렬: A=1 또는 B=1이면 GND 통로 생김
    |         |
   GND       GND
```

디지털 회로 설계 흐름(Design Flow)

Full-Custom 설계: 가장 정밀한 방법으로, 최고 성능을 낼 수 있습니다.

1. 논리 함수 정의 (예: Y = A·B)
2. 회로도(Schematic) 설계 — 트랜지스터를 어떻게 연결할지
3. 타이밍/전력 시뮬레이션 — SPICE 같은 툴로 동작 검증
4. 레이아웃(Layout) 설계 — 물리적인 칩 도면 그리기
5. DRC(Design Rule Check) — 제조 규칙 위반 검사
6. LVS(Layout vs Schematic) — 레이아웃이 회로도와 일치하는지 검사
7. 기생 성분 추출(Extraction) — 실제 배선의 저항/커패시턴스 측정
8. Post-layout 시뮬레이션 — 기생 성분 포함 최종 검증
9. Tape-out — 반도체 공장으로 제조 파일 전송

Semi-Custom 설계: 미리 만들어진 표준 셀 라이브러리(Standard Cell Library)를 사용합니다. 자동 배선 도구도 씁니다. 성능은 Full-Custom보다 낮지만 설계 시간이 훨씬 짧습니다. 대부분의 칩이 이 방식으로 만들어집니다.

추상화 3단계:

```
아키텍처 수준: 어떤 연산을 어떤 자원으로 구현?
        ↓
논리 수준: 논리 함수를 어떤 게이트로 구현?
        ↓
기하학 수준: 소자들을 물리적으로 어떻게 배치?
```

무어의 법칙(Moore's Law)과 그 한계

1965년 인텔 공동창업자 고든 무어(Gordon Moore)가 제시한 경험적 법칙: "집적회로의 트랜지스터 수는 약 2년마다 2배가 된다."

이 법칙 덕분에 수십 년간 칩은 계속 작아지고, 빨라지고, 저렴해졌습니다.

무어의 법칙이 흔들리기 시작한 두 가지 이유:

전압을 낮추기 어려워졌습니다. 트랜지스터가 작아질수록 원래는 동작 전압도 같이 낮춰야 합니다. 그런데 전압을 낮추면 트랜지스터가 OFF 상태일 때도 미세한 전류가 새어나오는 누설 전류 문제가 심해집니다. 현실에서는 전압을 충분히 낮추지 못했습니다.

전력 밀도가 폭증했습니다. 트랜지스터가 더 많아지고 더 빨리 동작하면서, 단위 면적당 발생하는 열이 급격히 증가했습니다. 이 문제가 너무 심해져서 인텔의 CPU 클럭 속도는 2004년경부터 더 이상 빠르게 올리기 어려워졌습니다. 이것을 **전력 위기(Power Crisis)**라고 합니다.

Intel의 Tick-Tock 모델 : 인텔은 "Tick(공정 미세화) → Tock(새 아키텍처)"을 번갈아 하는 전략을 썼는데, 10nm, 7nm 공정이 계속 지연되면서 이 리듬이 무너졌습니다.

극복 노력:

- FinFET: 트랜지스터를 3D로 세워서 누설 전류를 줄이는 구조. 22nm 공정부터 도입.
- 칩렛(Chiplet)과 이종 집적: 여러 칩을 하나의 패키지에 넣는 방식.
- More than Moore: TPU, NPU 같은 특화 프로세서로 성능을 높이는 방향.

현재(2022년 이후) SRAM 스케일링도 멈추고 있습니다. TSMC가 IEDM 2022에서 발표한 것처럼, 메모리 셀 크기 축소도 한계에 부딪히고 있습니다.

Ch2. MOSFET Device Characteristics

이 챕터는 왜 필요한가?

Ch1에서 MOSFET을 "Gate에 전압을 걸면 ON, 안 걸면 OFF가 되는 스위치"라고 이해했습니다. 그런데 설계자 입장에서는 더 많은 것을 알아야 합니다. 이 트랜지스터가 켜질 때 전류가 얼마나 흐르는지, 얼마나 빠르게 켜지는지, 작게 만들면 어떤 문제가 생기는지를 **정량적으로** 알아야 합니다.

이것이 Ch2의 목표입니다. 수식이 등장하지만, 각 수식이 무엇을 의미하는지 직관부터 잡고 들어갑니다.

에너지 밴드(Energy Band) — 전자의 에너지 지도

고체 안의 전자는 아무 에너지 값이나 가질 수 없습니다. 허용된 에너지 범위(밴드) 안에서만 존재할 수 있습니다. 그 사이에는 전자가 절대 존재할 수 없는 금지 구역(Forbidden Gap)이 있습니다.

```
에너지
  ↑
  │  [전도 밴드 (Conduction Band)]  Ec(최하단)
  │           ← 자유 전자가 있는 곳
  │  ← Eg (밴드갭) = 금지 구역 →
  │  [가전자 밴드 (Valence Band)]   Ev(최상단)
  │           ← 원자에 묶인 전자
  │  [Ei: 진성 페르미 레벨 - 순수 Si의 기준선]
```

- **전도 밴드(Conduction Band, Ec 이상)**: 원자에서 탈출해 자유롭게 이동할 수 있는 전자들이 있는 영역
- **가전자 밴드(Valence Band, Ev 이하)**: 원자에 묶여있는 전자들의 영역
- **밴드갭(Band Gap, Eg)**: 두 밴드 사이의 금지 구역. 실리콘(Si)은 약 1.12eV

밴드갭이 크면 전자가 자유로워지기 어렵고 절연체에 가깝습니다. 작으면 전도체에 가깝습니다.

페르미 레벨(Fermi Level, Ef) — "전자가 어디 있을 확률이 얼마인가"

페르미 레벨은 **전자가 그 에너지 상태에 존재할 확률이 정확히 50%인 에너지 값**입니다.

Fermi-Dirac 분포: f(E) = 1 / (1 + e^((E-Ef)/kT))

이 수식의 의미: - E가 Ef보다 훨씬 낮으면: f(E) ≈ 1 → 전자가 거의 확실하게 있음 - E = Ef이면: f(E) = 0.5 → 전자가 있을 확률 50% - E가 Ef보다 훨씬 높으면: f(E) ≈ 0 → 전자가 거의 없음

페르미 레벨이 왜 중요한가? **두 물질이 접촉하면 페르미 레벨이 같아질 때까지 전하가 이동합니다.** 이 원리가 PN 접합, MOS 구조 등 모든 반도체 소자 동작의 근본입니다.

캐리어 농도(Carrier Concentration)

N형과 P형에서 전자(n)와 정공(p)의 농도:

N형: n ≈ Nd (도너 농도), p = ni^2/Nd

P형: p ≈ Na (억셉터 농도), n = ni^2/Na

질량 작용 법칙(Mass Action Law): np = ni^2 — 항상 일정

이것은 "시소" 관계입니다. N형에서 전자를 많이 넣으면(n이 커지면), 정공은 그에 반비례해서 줄어듭니다(p가 작아짐). ni는 순수 실리콘의 진성 캐리어 농도로, 상온에서 약 $1.5 \times 10^{10}/cm^3$입니다.

도핑을 통해 전자나 정공 농도를 이 값보다 수백만 배 높일 수 있습니다. 이것이 반도체 소자 동작의 핵심입니다.

PN 접합(PN Junction)

MOSFET의 Source와 Drain은 기판과 PN 접합을 이룹니다. 이것을 이해해야 합니다.

N형과 P형을 붙이면 어떻게 될까요?

농도 차이로 인해 전자는 N→P 방향으로, 정공은 P→N 방향으로 확산(Diffusion)합니다. 캐리어들이 이동하면 뒤에 이온화된 불순물들이 남겨집니다. 이것이 **공핍 영역(Depletion Region)**입니다.

```
P형    [---|+++]   N형
       공핍  영역
       ←전기장→
```

이 전기장이 확산을 막아 평형 상태가 됩니다. 양쪽의 전위 차이를 **내장 전위(Built-in Potential, Vbi)**라고 합니다.

순방향 바이어스(Forward Bias): 양극에 양전압을 걸면 장벽이 낮아져 전류가 지수함수적으로 증가합니다.

$I = I_0 \times (e^{(qV/kT)} - 1)$

여기서 $kT/q \approx 26mV$ (상온 300K)

역방향 바이어스(Reverse Bias): 반대 전압을 걸면 장벽이 높아져 전류가 거의 흐르지 않습니다.

MOSFET에서의 의미: Source/Drain-기판 접합은 정상 동작 중 항상 역방향 바이어스 상태여야 합니다. 순방향이 되면 의도치 않은 전류가 기판으로 흘러 회로가 오동작합니다. 또한 역방향이라도 접합 커패시턴스가 생기는데, 이것이 회로 속도를 늦춥니다.

MOS 커패시터 — MOSFET의 핵심 구조

MOSFET의 Gate-산화막-반도체 구조는 그 자체로 하나의 커패시터처럼 동작합니다. Gate 전압에 따라 반도체 표면이 어떻게 변하는지가 핵심입니다.

① Flat Band 조건 (Vg = VFB)

에너지 밴드가 구부러지지 않고 평평한 상태입니다. 이것이 기준점입니다. VFB는 금속과 반도체의 일함수(Work Function) 차이로 결정됩니다.

② 공핍 조건 (Vg > VFB)

Gate에 양전압을 올리면, 전기장이 P형 기판의 정공들을 밀어냅니다. Gate 바로 아래 표면 근처에 정공이 없는 공핍층(Depletion Layer)이 형성됩니다.

공핍층 두께: $W_d = \sqrt{(2\varepsilon_{si} \times \varphi_s / qN_A)}$

Gate 전압이 올라갈수록 공핍층이 두꺼워집니다.

③ 반전 조건 (Vg > VTH)

Gate 전압을 계속 올리면 결정적인 순간이 옵니다. 표면의 전위가 충분히 낮아져서 전자들이 표면으로 끌려옵니다. P형 기판 표면에 N형 채널이 생기는 것입니다! 이것이 반전층(Inversion Layer)이고, 전류가 흐르는 통로입니다.

문턱 전압(VTH): 반전이 시작되는 최소 Gate 전압

$VTH = VFB + 2\varphi F + \sqrt{(2\varepsilon si \times qNA \times 2\varphi F)} / Cox$

여기서 $\varphi F = Ef - Ei$는 페르미 전위(Fermi Potential)입니다.

반전층의 전하량: $Qinv = Cox \times (Vg - VTH)$

이것이 MOSFET 전류 수식의 출발점입니다. **Gate 전압이 VTH를 넘는 만큼 채널 전하가 생긴다**는 매우 직관적인 관계입니다.

Cox는 단위 면적당 산화막 커패시턴스입니다: $Cox = \varepsilon ox / tox$

산화막이 얇을수록 Cox가 커집니다. 이것이 왜 반도체 공정에서 산화막을 점점 얇게 만들려 하는지의 이유입니다. Cox가 클수록 같은 전압으로 더 많은 전하를 만들 수 있어 더 많은 전류가 흐릅니다.

MOSFET I-V 특성 수식 유도

이제 이 챕터의 핵심입니다. MOSFET에 흐르는 전류가 어떻게 결정되는지 수식으로 이해합니다.

채널을 X방향(Source → Drain)으로 봤을 때, 위치 x에서:

1. 채널 전하: $Qi(x) = Cox \times [Vgs - Vt - V(x)]$
2. 전류: $Ids = W \times Qi(x) \times \mu n \times |E(x)|$
3. 전기장: $E(x) = -dV/dx$

핵심 아이디어: V(x)는 x 위치에서의 전위입니다. Source 쪽(x=0)은 V=0, Drain 쪽(x=L)은 V=Vds입니다. Drain으로 갈수록 전위가 높아져서 채널 전하가 줄어듭니다. 이 세 식을 연립해서 적분하면:

선형 영역 (Linear Region): Vds < Vgs - Vt

$Ids = \mu nCox \times (W/L) \times [(Vgs - Vt) \times Vds - Vds^2/2]$

- μn: 전자 이동도(Electron Mobility). 전자가 전기장 안에서 얼마나 잘 움직이는지. 단위 $cm^2/V{\cdot}s$
- Cox: 단위 면적당 산화막 커패시턴스
- W/L: 채널 너비 대 길이 비율. 클수록 전류 많이 흐름

Vds가 매우 작을 때(Vds^2 항 무시):

$Ids \approx \mu nCox \times (W/L) \times (Vgs - Vt) \times Vds$

Ids가 Vds에 선형 비례합니다. 저항처럼 동작합니다. 트랜지스터가 “켜진 저항”으로 쓰이는 상태입니다.

포화 영역 (Saturation Region): Vds > Vgs - Vt

Vds를 계속 키우면 Drain 쪽 채널 전하가 0이 되는 순간이 옵니다. Vds = Vgs - Vt인 점에서 **핀치오프(Pinch-off)**가 발생합니다.

이 상태에서 Vds를 더 올려도 전류는 더 이상 증가하지 않고 포화됩니다:

$Ids = (1/2) \times \mu nCox \times (W/L) \times (Vgs - Vt)^2$

Ids가 Vds와 무관해집니다. Gate 전압으로만 전류를 제어합니다. 디지털 회로에서 가장 중요한 동작 영역입니다.

이 수식은 소프트웨어 관점에서 이렇게 해석할 수 있습니다. 파이프라인의 처리량(Throughput)이 가장 느린 병목 단계로 제한되는 것처럼, MOSFET 전류도 Gate 전압이라는 “제한 조건”에 의해 결정됩니다.

채널 길이 변조 (Channel Length Modulation)

이론적으로 포화 영역에서 Ids는 Vds와 무관해야 합니다. 그런데 현실에서는 Vds가 올라갈수록 Ids가 조금씩 증가합니다.

이유: Vds가 커질수록 핀치오프 점이 Source 쪽으로 이동하여 유효 채널 길이 L이 조금씩 짧아집니다. L이 짧아지면 W/L이 커지고 전류가 증가합니다.

$Ids = (1/2) \times \mu nCox \times (W/L) \times (Vgs - Vt)^2 \times (1 + \lambda Vds)$

λ(람다)가 채널 길이 변조 계수입니다. 채널이 짧을수록 이 효과가 더 강합니다.

단채널 효과 (Short Channel Effects)

공정이 발전하면서 트랜지스터가 점점 작아지자 기존 이론에서 벗어난 현상들이 나타납니다.

속도 포화 (Velocity Saturation)

지금까지 전자의 속도는 전기장에 비례한다고 가정했습니다: $v = \mu n \times E$

그런데 채널이 짧아지면 전기장이 강해지고, 전자 속도가 **최대 속도(포화 속도, vsat)**에 도달합니다. 더 강한 전기장을 걸어도 속도가 늘지 않습니다.

결과: 장채널에서 $Ids \propto (Vgs-Vt)^2$이지만, 단채널에서는 $Ids \propto (Vgs-Vt)$로 선형에 가까워집니다. 성능 향상 효율이 줄어드는 것입니다.

```
전자속도(v)
    |      한계선(vsat) ──────────────────
    |    /  실제 특성
    |   /
    |  /   기울기 = μn
    |/
    └──────────────────→ 전기장(E)
              Ec (임계 전기장)
```

문턱 전압 roll-off와 DIBL

채널이 짧아지면 VTH가 변합니다.

VTH roll-off: Source/Drain의 공핍층이 채널 공핍층의 일부를 분담합니다. 그러면 Gate가 공핍층을 형성하는 데 필요한 전하가 줄어들어 VTH가 낮아집니다.

DIBL (Drain-Induced Barrier Lowering): Drain 전압이 커지면 Drain 쪽 전위가 Source 쪽 채널 장벽에 영향을 미쳐 VTH를 추가로 낮춥니다. 채널이 짧을수록 Drain의 영향이 더 멀리(Source까지) 미칩니다.

두 효과 모두 채널 길이가 짧아질수록 심해집니다.

서브스레숄드 전류 (Subthreshold Current)

Vgs < VTH, 즉 트랜지스터가 "꺼져있어야" 하는 상태에서도 미세한 전류가 흐릅니다. 이것이 서브스레숄드 전류입니다.

Ids ∝ e^(qVgs/nkT)

서브스레숄드 스윙(S): 전류를 10배(1 decade) 바꾸는 데 필요한 Vgs 변화량

S = (kT/q) × ln(10) × n ≥ 60 mV/decade

상온(300K)에서 이론적 최솟값은 **60 mV/decade**입니다. 현실에서는 60~100 mV/decade입니다.

```
Ids (로그 스케일)
  │                        / 포화
  │                      /
  │                    /    60~100 mV/decade
  │                  /  ← 서브스레숄드 전류
  │_______________/____________ Vgs
                Vth
```

왜 중요한가? S가 작을수록 트랜지스터를 빠르게 ON/OFF할 수 있고, OFF 상태 누설이 줄어듭니다. **60 mV/decade는 현재 CMOS 기술의 근본적인 한계**입니다. 이것은 상온 열에너지(kT)에 의한 것으로, 온도를 낮추지 않는 한 이 장벽을 넘을 수 없습니다. 이를 돌파하기 위해 터널 FET(Tunnel FET), 네거티브 커패시턴스 FET 등의 연구가 활발합니다.

MOSFET 커패시턴스

디지털 회로에서 속도는 커패시턴스를 얼마나 빨리 충방전하느냐로 결정됩니다.

Gate 커패시턴스(CGC): 동작 영역에 따라 달라집니다.

- OFF 상태(차단 영역): Gate와 채널 사이에 공핍층 → Cox와 Cd가 직렬 → 총 커패시턴스 < Cox
- ON 상태(반전 영역): 채널이 Gate 바로 아래 형성 → 총 커패시턴스 ≈ Cox (최대)
- 포화 영역: 약 (2/3) × Cox × W × L

디지털 설계에서 중요한 영역은 포화 영역과 차단 영역입니다.

Source/Drain 접합 커패시턴스: Source와 Drain이 기판과 이루는 역방향 바이어스 PN 접합에 생기는 커패시턴스입니다. 바닥면과 측벽 두 부분으로 나뉩니다.

이 커패시턴스들은 트랜지스터가 스위칭할 때마다 충방전되므로, 동적 전력 소비와 지연의 직접적인 원인입니다.

Ch3. CMOS Circuit Basics

이상적인 디지털에서 현실로

지금까지는 논리 1 = VDD, 논리 0 = GND 라고 가정했습니다. 예를 들어 1 = 1V, 0 = 0V. 그런데 현실에서는 칩 위의 수십억 트랜지스터들이 동시에 스위칭하고, 전선들이 서로 나란히 달립니다. 이로 인해 신호에 잡음(Noise)이 섞입니다.

만약 1이어야 할 신호가 0.9V가 되거나 0.55V가 된다면? 수신하는 게이트는 이것을 1로 봐야 할까요 0으로 봐야 할까요?

노이즈의 원인 세 가지:

1. **유도성 결합(Inductive Coupling)**: 주변 배선의 전류 변화가 자기장을 만들고, 이것이 근처 신호선에 전압을 유도합니다.

2. **용량성 결합(Capacitive Coupling)**: 나란히 달리는 배선들이 커패시터처럼 작용해 한 쪽 신호가 다른 쪽에 영향을 줍니다.

3. **전원/접지 노이즈**: 많은 게이트가 동시에 스위칭하면 전원선 전압이 순간적으로 출렁입니다.

현실에서 완벽한 0과 1은 없습니다.

노이즈 마진(Noise Margin)

얼마나 오염되어도 정상적으로 인식할 수 있는가?를 정량화한 것이 노이즈 마진입니다.

4개의 기준 전압:

- **VOH (Output High)**: 출력이 HIGH일 때 보장하는 최솟값
- **VOL (Output Low)**: 출력이 LOW일 때 보장하는 최댓값
- **VIH (Input High)**: 이 전압 이상이면 입력을 HIGH로 인식하는 최솟값
- **VIL (Input Low)**: 이 전압 이하이면 입력을 LOW로 인식하는 최댓값

```
전압
VOH ──────────────── ↑
     출력 HIGH 영역   │ NMH = VOH - VIH
VIH ─ ─ ─ ─ ─ ─ ─  ↓
     불확정 영역  ← 이 사이는 어느 쪽인지 보장 못 함
VIL ─ ─ ─ ─ ─ ─ ─  ↑
     출력 LOW 영역   │ NML = VIL - VOL
VOL ──────────────── ↓
```

NMH = VOH - VIH: HIGH 신호가 최대 NMH만큼 깎여도 다음 게이트가 HIGH로 인식합니다.

NML = VIL - VOL: LOW 신호가 최대 NML만큼 올라와도 다음 게이트가 LOW로 인식합니다.

이상적 인버터: NMH = NML = VDD/2

전압 전달 특성 (VTC, Voltage Transfer Characteristic)

인버터에 입력 전압을 0V에서 VDD까지 올리면서 출력이 어떻게 변하는지 그린 그래프입니다.

```
Vout
VDD ──────┐  A      B   C    D      E
          │  NMOS off → NMOS 약 → 전환 → PMOS 약 → PMOS off
          │        \
          │         \  ← VTC 곡선 (기울기가 가파른 부분이 전환 영역)
          │          \
   0 ─────└──────────────────→ Vin
```

```
0  Vtn   VDD/2   VDD
```

5개 구간: - A: NMOS OFF, PMOS 강하게 ON → Vout = VDD - B: NMOS 약하게 켜짐 → Vout 서서히 내려감 - C: 둘 다 포화 영역, 기울기 최대 → 급격한 전환 - D: PMOS 약해짐 → Vout 계속 내려감 - E: PMOS OFF, NMOS 강하게 ON → Vout = GND

VIL과 VIH: VTC에서 기울기(dVout/dVin)가 정확히 -1인 두 점입니다.

왜 -1인 점이 기준인가? 그 점에서는 입력의 작은 변화가 출력에 같은 크기의 변화를 만듭니다. 기울기 절댓값이 1보다 크면 출력 변화가 입력 변화보다 크다는 뜻으로, 그 바깥 영역에서는 신호가 "증폭"되어 명확한 0이나 1로 수렴합니다.

재생 특성 (Regenerative Characteristics)

디지털 시스템의 놀라운 특성: 노이즈로 오염된 신호가 게이트를 거칠수록 스스로 깨끗해집니다.

- Vin < VIL이면: 버퍼를 반복 통과할수록 출력이 VOL에 수렴 (신호 복원)
- Vin > VIH이면: 버퍼를 반복 통과할수록 출력이 VOH에 수렴 (신호 복원)
- VIL < Vin < VIH이면: 어느 쪽으로 수렴할지 알 수 없습니다 (불확정)

좋은 게이트 조건: VTC의 기울기 절댓값이 1보다 훨씬 커야 합니다. 그래야 불확정 영역이 좁아지고 노이즈 마진이 넓어집니다.

동적 전력(Dynamic Power)

CMOS 인버터의 출력에는 항상 부하 커패시턴스(CL)가 연결되어 있습니다. 출력이 0→1로 바뀔 때 CL을 VDD까지 충전해야 하고, 1→0으로 바뀔 때 GND까지 방전해야 합니다.

에너지 계산:

0→1 전환 시 전원이 공급하는 에너지: $E = CL \times VDD^2$

이 중 절반($\frac{1}{2}CL\ VDD^2$)은 CL에 저장, 나머지 절반은 PMOS에서 열로 소모됩니다.

1→0 전환 시 CL에 저장된 에너지($\frac{1}{2}CL\ VDD^2$)가 NMOS에서 열로 소모됩니다.

주파수 f로 동작하면:

$$\mathbf{P = CL \times VDD^2 \times f}$$

이것이 동적 전력 소비의 핵심 공식입니다.

왜 VDD 낮추기가 가장 효과적인가?

VDD가 제곱으로 들어갑니다. VDD를 절반으로 낮추면 전력은 1/4이 됩니다. 주파수를 절반으로 낮추면 전력은 1/2만 줄지만, VDD를 절반으로 낮추면 전력이 1/4로 줄어듭니다.

이것이 무어의 법칙에서 소비 전력 위기의 근본 원인입니다. 트랜지스터를 작게 만들면서 트랜지스터 수와 주파수가 늘어났는데, VDD를 충분히 낮추지 못하면 전력이 폭증합니다.

정적 누설 전력 (Static Leakage Power)

CMOS의 장점이 정적 전력 소비가 0이라고 했는데, 현실은 다릅니다.

Ch2에서 배운 서브스레숄드 전류가 있습니다. OFF 상태인 트랜지스터에서도 지수함수적으로 미세한 전류가 흐릅니다.

P_leak = I_leak × VDD

트랜지스터 하나당 누설은 미미하지만, 수백억 개를 합산하면 무시 못 할 전력이 됩니다. 현대 고성능 프로세서에서 누설 전력이 전체 전력의 상당 부분을 차지하는 이유입니다.

CMOS 인버터 지연 모델

MOSFET이 켜진 상태를 단순히 ON 저항으로 모델링합니다.

```
Pull-up (0→1 출력):    Pull-down (1→0 출력):
  VDD                     VDD
   |                       |
  [Rp] ← PMOS ON 저항  (차단됨)
   |                       |
   +── Vout                +── Vout
   |                       |
  [CL]                   [Rn] ← NMOS ON 저항
   |                       |
  GND                    [CL]
                           |
                          GND
```

RC 충방전 회로에서 50%에 도달하는 시간:

- **tpLH = 0.69 × Rp × CL** (출력 0→1, Pull-up 지연)
- **tpHL = 0.69 × Rn × CL** (출력 1→0, Pull-down 지연)
- **tp = 0.69 × CL × (Rp + Rn) / 2** (평균 전파 지연)

0.69 = ln(2)입니다. RC 회로에서 50%에 도달하는 시간이 0.69 × RC이기 때문입니다.

지연을 줄이려면: 1. CL을 줄인다 (다음 단 게이트를 작게, 배선을 짧게) 2. R을 줄인다 (트랜지스터 W를 크게 → 전류 많이 흐름 → 저항 낮아짐)

그런데 W가 커지면 트랜지스터 자체의 커패시턴스도 커집니다. 전단 게이트 입장에서 CL이 커지는 것입니다. **자신의 속도를 위해 커지면 이전 단에 부담을 주는 트레이드오프**가 있습니다.

CMOS 인버터 사이징

왜 PMOS를 NMOS의 2배 크게 만드는가?

정공의 이동도(μp)가 전자의 이동도(μn)보다 약 2배 낮습니다. 이동도가 낮으면 같은 조건에서 전류가 더 적게 흘러 ON 저항이 더 높습니다.

Pull-up(PMOS)과 Pull-down(NMOS) 지연을 같게 만들려면 PMOS의 ON 저항을 NMOS 수준으로 낮춰야 합니다.

저항 R ∝ 1/W이므로, PMOS W를 2배로 하면 PMOS 저항이 절반이 되어 NMOS와 비슷해집니다.

```
          VDD
           |
```

```
[PMOS, W=2]   ← 2배 크게
    |
    Y
    |
[NMOS, W=1]
    |
   GND
```

NAND2 게이트 사이징:

NMOS 2개가 직렬 → 저항이 2배 → 인버터와 같은 속도 내려면 NMOS W를 2배로 PMOS 2개는 병렬 → 저항이 절반 → 인버터 PMOS 저항 맞추려면 그대로 2W

결과: PMOS W=2, NMOS W=2

일반 규칙: **직렬로 연결된 트랜지스터 N개 → 각 W를 N배로**

인버터 체인 최적 사이징

작은 드라이버가 매우 큰 부하를 구동할 때, 인버터 하나보다 여러 단으로 나눠서 키우는 게 더 빠릅니다.

입력 Cin=Wi, 출력 CL=64Wi인 경우, 3단으로 나누면:

각 단의 크기 비율이 동일해야 최소 지연: α = (64)^(1/3) = 4

```
Wi → [×1] → 4Wi → [×4] → 16Wi → [×16] → 64Wi
```

왜 동일 비율이 최적인가? 어느 한 단이 특히 느리면 그것이 병목이 됩니다. 모든 단의 지연이 같아야 전체 지연이 최소화됩니다.

최적 비율 α ≈ e ≈ 2.7 (수학적으로 유도), 실용적으로는 3~4단이 적당합니다.

Ch4. Logical Effort

이 챕터는 왜 필요한가?

NAND, NOR, 복합 게이트들이 섞인 경로가 있을 때, SPICE 시뮬레이션 없이 "이 경로에서 최적 게이트 크기는 무엇인가?"를 손으로 빠르게 계산하는 방법론이 Logical Effort입니다.

마치 알고리즘 복잡도 분석에서 정확한 실행 시간 측정 없이 O(n), O(n log n)으로 비교하는 것처럼, Logical Effort는 게이트의 "어려움"을 수치로 나타내어 빠르게 설계 결정을 내릴 수 있게 합니다.

기생 커패시턴스 포함 지연

Ch3에서 배운 지연은 부하 커패시턴스만 고려했습니다. 현실에서는 트랜지스터 자체의 기생 커패시턴스(접합 커패시턴스)도 있습니다.

인버터를 예로 들면:

```
VDD
 |
```

```
[PMOS, 2W] → 접합 커패시턴스 Cjp
    |
    Y ── CL (다음 단 부하)
    |
[NMOS, W]  → 접합 커패시턴스 Cjn
    |
   GND
```

총 부하 = CL + Cjp + Cjn = CL + Cpar

지연 공식 정규화:

td = td0 × (γ + f)

- **td0**: 입출력 크기가 같은 인버터의 기준 지연
- **γ (감마)** = Cpar/Cin: 기생 커패시턴스 계수. 인버터에서 γ ≈ 1
- **f (fanout)** = Cout/Cin: 전기적 팬아웃. 내가 구동하는 부하가 내 입력 커패시턴스의 몇 배인가

γ는 부하가 0이어도 기생 커패시턴스 때문에 발생하는 최소 지연을 나타냅니다. 트랜지스터가 아무리 크더라도 이 지연은 피할 수 없습니다.

기생 커패시턴스가 있어도 최적 인버터 체인 설계는 변하지 않습니다. 각 단의 effort(팬아웃)가 동일할 때 전체 지연이 최소라는 결론은 그대로 유지됩니다.

Logical Effort (LE)의 정의

Logical Effort(논리적 노력)는 이 질문에 답합니다:

"같은 출력 전류를 내기 위해, 이 게이트는 인버터에 비해 입력 커패시턴스가 몇 배나 더 필요한가?"

LE = Cin(해당 게이트) / Cin(동일 전류를 내는 인버터)

인버터의 LE = 1이 기준입니다.

인버터 LE 계산:

인버터 입력에 연결되는 PMOS(2W)와 NMOS(W)의 게이트 커패시턴스를 각각 Cg라 하면:

Cin(인버터) = PMOS(2W)의 게이트 + NMOS(W)의 게이트 = 2Cg + Cg = 3Cg

LE(인버터) = 3Cg / 3Cg = 1

NAND2 LE 계산:

NAND2의 각 입력에는 PMOS(2W)와 NMOS(2W)가 연결됩니다. (NMOS 직렬이므로 2W 필요)

Cin(NAND2) = 2Cg + 2Cg = 4Cg

LE(NAND2) = 4Cg / 3Cg = 4/3 ≈ 1.33

NOR2 LE 계산:

NOR2의 각 입력에는 PMOS(4W)와 NMOS(W)가 연결됩니다. (PMOS 직렬이므로 4W 필요)

Cin(NOR2) = 4Cg + Cg = 5Cg

LE(NOR2) = 5Cg / 3Cg = 5/3 ≈ 1.67

NOR > NAND인 이유: NOR에서는 PMOS가 직렬로 연결됩니다. PMOS는 이동도가 낮아서 같은 전류를 내려면 더 크게 만들어야 합니다. PMOS를 더 크게 만들면 입력 커패시턴스가 커집니다. 이것이 CMOS 회로에서 NOR보다 NAND를 선호하는 이유입니다.

게이트별 LE 요약:

게이트	Cin	LE
인버터	3Cg	1
NAND2	4Cg	4/3
NOR2	5Cg	5/3
NAND3	5Cg	5/3
NOR3	7Cg	7/3

LE를 포함한 지연 공식

td = td0 × (p + LE × f)

- p: 기생 지연 (인버터=1, NAND2=2, NOR2=2, ...)
- LE: 논리적 노력
- f: 전기적 팬아웃 = Cout/Cin
- LE × f를 "Effort"라고 부릅니다. 이것이 각 단에서의 실질적 부담입니다.

예시: NAND2 게이트에서 f=4, γ=1이면

td = td0 × (2 + (4/3) × 4) = td0 × (2 + 5.33) = 7.33 × td0

경로(Path) Logical Effort

여러 게이트가 연결된 경로 전체를 분석합니다.

```
1 → [INV] → X → [NAND2] → Y → [NOR2] → Z → [INV] → 256
    LE=1        LE=4/3        LE=5/3       LE=1
    p=1         p=2           p=2          p=1
```

Path Logical Effort (G): 각 단 LE의 곱

G = 1 × 4/3 × 5/3 × 1 = 20/9 ≈ 2.22

Path Electrical Effort (EF): 전체 전기적 팬아웃

EF = Cout/Cin = 256/1 = 256

Path Parasitic Delay (P): 기생 지연 합

P = 1 + 2 + 2 + 1 = 6

Path Effort (PE) = G × B × EF

(B는 분기 노력, 분기 없으면 B=1)

PE = 2.22 × 1 × 256 = 568

최소 경로 지연: 각 단의 effort(LE×f)가 동일할 때 최소화됩니다.

D = N × td0 × PE^(1/N) + P × td0

N=4단이면: D = 4 × td0 × 568^(0.25) + 6 × td0 ≈ 4 × 4.88 + 6 = 25.5 td0

이것이 이 챕터에서 제시한 예시 "Path Delay ≈ 26td0"의 계산 결과입니다.

설계 6단계 절차

1. **PE 계산**: PE = G × B × EF
2. **최적 단수 결정**: N = $\log_4$(PE)
3. **N단 경로 스케치**
4. **최소 지연 추정**: D = N × td0 × PE^(1/N) + P × td0
5. **단당 최적 effort**: $\hat{f}$ = PE^(1/N)
6. **뒤에서 앞으로 크기 결정**: Cin_i = (Cout_i × LE_i) / $\hat{f}$

왜 뒤에서 앞으로 계산하는가? 최종 출력 부하는 이미 고정되어 있습니다. 거기서부터 역방향으로 각 단의 입력 커패시턴스를 구하면 게이트 크기가 자연스럽게 결정됩니다.

분기 노력 (Branching Effort, B)

신호가 두 방향으로 갈라질 때(브랜치):

```
─→ [게이트] ─→ 64Wi (on-path)
              └→ 16Wi (off-path)
```

b = (Con-path + Coff-path) / Con-path = (64 + 16) / 64 = 1.25

분기가 있으면 off-path도 구동해야 하므로 부담이 늘어나고 PE가 증가합니다.

LE의 한계

LE는 강력한 도구지만 한계가 있습니다:

1. **인터커넥트 무시**: 긴 배선의 저항/커패시턴스를 고려하지 않음 → Ch5에서 다룸
2. **지연만 최적화**: 전력이나 면적은 고려하지 않음
3. **균일 분기 가정**: on-path와 off-path 비율이 일정하다고 가정
4. **입력 슬루 무시**: 입력 신호가 이상적으로 빠르게 변한다고 가정

Ch5. Interconnect Wire

왜 배선이 문제인가?

현대 칩에서는 트랜지스터 지연보다 **배선 지연이 더 큰 경우**도 많습니다. Logical Effort의 가장 큰 한계인 "인터커넥트 무시"를 이 챕터에서 다룹니다.

배선이 긴 경우 세 가지 문제가 생깁니다:

- **저항(R)**: 배선이 길수록 커짐 → 전력 소모, 지연 증가
- **커패시턴스(C)**: 배선이 길수록 커짐 → 지연 증가, 전력 소모
- **인덕턴스(L)**: 온칩에서 보통 무시 가능

배선 기하학적 구조 (Wire Geometry)

```
   ←W(폭)→
   ┌─────┐  ↑H(두께)
   │ 금속 │  ↓            Pitch = W + S
   └─────┘               AR(Aspect Ratio) = H/W
← S(간격)→
──────────────── (절연체, 두께 T)
```

AR (Aspect Ratio): H/W. 배선의 세로 대 가로 비율.

- 옛날: AR << 1 (납작한 배선)
- 현재: AR이 점점 커지는 추세

왜 AR을 키우는가? 배선을 좁게(W↓) 만들면서도 저항이 너무 커지지 않도록 높이(H↑)를 유지합니다. 단면적을 유지해서 저항을 관리하는 겁니다.

배선 저항 (Wire Resistance)

R = ρ × L / (W × H) = (ρ/H) × (L/W) = **R□ × (L/W)**

R□ (Sheet Resistance, 면저항) = ρ/H, 단위: Ω/□(오옴 퍼 스퀘어)

L/W는 위에서 봤을 때 정사각형이 몇 개 들어가는가입니다.

```
L=3W인 배선을 위에서 보면:
┌──┬──┬──┐
│ □ │ □ │ □ │  → 3개 정사각형 → R = 3 × R□
└──┴──┴──┘
```

핵심: 배선의 저항은 절대 크기가 아니라 **모양(비율)**으로 결정됩니다. 같은 비율이면 크기에 관계없이 저항이 같습니다.

구리(Cu): ρ ≈ 1.7 μΩ·cm → 현대 칩 배선 재료 (알루미늄 Al: 2.7 μΩ·cm보다 낮음)

배선 커패시턴스 (Wire Capacitance)

평행판 커패시턴스: C = εdi × WL / T

εdi: 유전체(절연층)의 유전율, T: 절연층 두께

유전율이 낮을수록 커패시턴스가 줄어듭니다. 그래서 배선 절연체로 **Low-k 유전체**가 필요합니다.

반대로 Gate 산화막은 Cox를 크게 하기 위해 **High-k 유전체**가 필요합니다. 배선과 Gate가 서로 반대 요구를 하는 것입니다.

프린징 커패시턴스: 배선이 얇아지고 키가 커지면(AR↑), 배선 측면에서도 전기장이 퍼져나갑니다. 이 "프린징 효과"가 AR이 클수록 무시할 수 없게 됩니다.

커플링 커패시턴스(Cc): 나란히 달리는 두 배선 사이의 커패시턴스.

역상 스위칭(한 쪽 올라가고 한 쪽 내려갈 때): **밀러 효과**로 유효 커패시턴스가 2배

Ceff = Cself + 2Cc

동상 스위칭(둘 다 같은 방향): Ceff = Cself (상쇄)

해결책: 쉴드(Shield) 삽입

크리티컬한 신호선 양쪽에 GND 또는 VDD 배선을 넣습니다. 쉴드는 스위칭하지 않으므로 밀러 효과 없이 노이즈가 차단됩니다. 대신 배선 면적이 늘어나는 트레이드오프가 있습니다.

Elmore 지연 (Elmore Delay)

배선을 RC가 분포된 구조로 모델링할 때, 지연을 효율적으로 계산하는 공식입니다.

정의: $TDi = \Sigma k\ Rki \times Ck$

- Ck: 각 노드의 커패시턴스
- Rki: 소스→i 경로와 소스→k 경로의 **공통 경로 저항**

직관적 이해: 소스에서 노드 i까지 가는 경로 위의 저항들 중, 소스에 가까운 저항일수록 더 많은 커패시턴스를 충전해야 합니다. 따라서 소스에 가까운 저항이 지연에 더 크게 기여합니다.

```
예시:
소스 —R1— 노드1(C1) —R2— 노드2(C2) —R3— 노드3(C3)

TDi(노드2까지) = R1×(C1+C2+C3) + R2×(C2+C3)
                  ↑                  ↑
              소스에 가까워          더 적은 C를 담당
              모든 C를 담당
```

Elmore 지연의 수학적 배경: 임펄스 응답의 1차 모멘트(평균 시간)입니다. 계단 함수 응답의 50% 지점을 직접 구하기 어렵기 때문에, 더 다루기 쉬운 임펄스 응답의 평균 시간으로 근사합니다. RC 트리에서 이 근사가 매우 잘 맞습니다.

배선 RC 모델 비교

긴 배선은 저항과 커패시턴스가 분포된 구조입니다. 이를 단순화하는 세 가지 방법:

```
L형:   R → C            지연 = RC    ← 실제보다 2배 과대평가 × 사용 금지
π형:  C/2 → R → C/2    지연 = RC/2  ✅
T형:  R/2 → C → R/2    지연 = RC/2  ✅
분산RC (실제):           지연 = RC/2 (기준)
```

분산 RC 지연이 RC/2인 이유:

배선을 N개 구간으로 나누면 각 구간은 R/N과 C/N입니다.

Elmore 지연 = $(R/N)(C/N)(1 + 2 + \ldots + N) = RC \times N(N+1)/(2N^2) \to RC/2\ (N\to\infty)$

긴 배선 지연은 L^2에 비례한다

인버터 → π형 배선 모델 → 수신 인버터 구조에서:

$TD \approx Rtr \times (Cw + Ctr) + Rw \times (Cw/2 + Ctr)$

여기서 $Rw \propto L$, $Cw \propto L$이므로:

$$\mathbf{twire \approx Rw \times Cw / 2 \propto L^2}$$

배선이 2배 길어지면 지연이 4배가 됩니다. 이것이 긴 배선이 치명적인 이유입니다.

리피터 삽입 (Repeater Insertion)

L^2에 비례하는 배선 지연을 L에 비례하도록 바꾸는 해결책이 리피터(Repeater) 삽입입니다.

긴 배선을 N개 구간으로 나누고 각 구간 사이에 인버터(리피터)를 삽입합니다.

```
긴 배선 (지연 ∝ L²):
[드라이버]──────────────────────────────[수신단]

리피터 삽입 (지연 ∝ L):
[드라이버]──[REP]──[REP]──[REP]──[수신단]
            L/N    L/N    L/N
```

각 구간의 배선 지연: $\propto (L/N)^2 = L^2/N^2$

N개 구간의 총 지연: $N \times L^2/N^2 = L^2/N$ → N이 커질수록 총 지연 감소

최적 리피터 크기 (Wopt)와 **최적 리피터 수 (Nopt)**:

지연을 W와 N에 대해 미분하여 최소화:

$Wopt = \sqrt{(Rinv \times Cw / Rw \times Cinv)}$

$Nopt = \sqrt{(Rw \times Cw / 2Rinv \times Cinv)}$

트레이드오프: 리피터를 삽입하면 인버터가 추가되어 동적 전력이 늘어납니다. 지연 최적화와 에너지-지연 곱(EDP) 최적화의 최적 리피터 크기가 다릅니다.

배선 스케일링 이슈

공정이 발전할수록 트랜지스터는 작아집니다. 그런데 배선은 두 종류가 있습니다.

로컬 배선 (Local Wire): 같은 블록 안에서 인접 게이트들을 연결. 트랜지스터가 작아질수록 함께 짧아집니다. 공정 발전과 함께 지연이 개선됩니다.

글로벌 배선 (Global Wire): 서로 멀리 떨어진 블록을 연결. 칩의 물리적 크기 자체가 줄어들지 않기 때문에, 트랜지스터가 작아져도 이 배선들의 길이는 줄어들지 않습니다.

공정 발전의 역설: 공정이 발전할수록 배선이 가늘어져서(W↓) 저항(R□↑)이 오히려 증가할 수 있습니다. 따라서 글로벌 배선 지연은 공정 발전과 함께 개선되지 않고, 오히려 악화될 수 있습니다.

이것이 글로벌 배선이 리피터 삽입의 주요 대상인 이유입니다.

Ch6. Clocking Elements

이 챕터는 왜 필요한가?

지금까지 배운 모든 회로는 조합 논리(Combinational Logic)입니다. 입력이 들어오면 일정 지연 후 출력이 나오는, 기억이 없는 회로입니다. 실제 디지털 시스템은 현재 상태를 저장하고, 클럭에 맞춰 다음 상태로 넘어가는 순차 논리(Sequential Logic)가 필요합니다.

그 핵심 소자가 래치(Latch)와 플립플롭(Flip-Flop)입니다.

D-Latch — "투명한 창문"

D-Latch는 CLK 신호에 따라 두 가지 모드로 동작합니다.

CLK = 1 (투명 모드, Transparent)

입력 D가 그대로 출력 Q로 통과합니다. D가 바뀌면 Q도 즉시 바뀝니다.

CLK = 0 (불투명 모드, Opaque)

창문이 닫힙니다. D가 어떻게 바뀌든 Q는 CLK가 1이었을 때의 마지막 값을 유지합니다.

```
타이밍:
CLK  ┐   ┌───┐   ┌
     └───┘   └───┘

D    ──A──┬──B──┬──C──
Q    ──A──┴──B──┘  B유지
```

레벨 감지 래치(Level-Sensitive Latch) 또는 투명 래치(Transparent Latch)라고도 합니다.

D 플립플롭 — "셔터 순간의 사진"

D 플립플롭은 두 개의 D-Latch를 직렬 연결한 것입니다. 앞쪽을 마스터(Master, L1), 뒤쪽을 슬레이브(Slave, L2)라 하고, 두 래치에 반대 극성의 CLK를 공급합니다.

```
D ─→ [L1: CLKbar] ─→ N1 ─→ [L2: CLK] ─→ Q
     (CLK=0: 투명)        (CLK=1: 투명)
     (CLK=1: 불투명)      (CLK=0: 불투명)
```

CLK=0일 때: L1(마스터)이 투명 → D가 N1으로 통과, L2(슬레이브)는 불투명 → Q 유지

CLK=1일 때: L1(마스터)이 불투명 → N1에 저장된 이전 D값이 고정됨, L2(슬레이브)가 투명 → N1의 값이 Q로 출력

결과: Q는 CLK가 **0→1로 상승하는 에지 순간**의 D값만 캡처합니다. 이것이 에지 트리거(Edge-Triggered) 동작입니다.

```
CLK  ──┐  ┌──┐  ┌──
       └──┘  └──┘

D    ──A──┬──────┬──B──┬──C──
          │      │     │
Q    ──────A──────────B────  ← 상승 에지 순간의 D값만
```

에지 트리거 플립플롭(또는 마스터-슬레이브 FF)이 가장 널리 쓰이는 클로킹 소자인 이유: **명확하고 단순한 동기화**를 제공합니다.

왜 래치가 아니라 플립플롭을 쓰는가?

세 개의 데이터 흐름 A, B, C가 있고, 각각의 첫 번째 연산(A0, B0, C0)이 끝나는 타이밍이 다릅니다. 래치로 동기화를 시도하면:

CLK가 HIGH인 구간(투명 구간)이 충분히 길 경우, **빠른 경로의 데이터(B0)가 래치를 두 번 통과**해 버립니다. 이것이 더블 샘플링(Double Sampling) 문제입니다. A0, C0는 한 번만 샘플링되는데 B0는 두 번 샘플링되어 데이터 동기화가 깨집니다.

플립플롭은 에지 순간에만 캡처하므로, 데이터가 아무리 빨리 도착해도 다음 에지 전까지는 절대 통과하지 않습니다. 완벽한 동기화가 됩니다.

Setup Time & Hold Time ★★★

플립플롭이 데이터를 정확히 캡처하려면 입력 D가 일정 시간 동안 안정적이어야 합니다.

Setup Time (tsetup): 클럭 상승 에지 이전에 최소 tsetup만큼 D가 안정되어 있어야 합니다.

이유: 마스터 래치(L1)가 CLK=1이 되기 전에 D값을 충분히 캡처해야 하기 때문입니다.

Hold Time (thold): 클럭 상승 에지 이후에도 최소 thold 동안 D가 안정적으로 유지되어야 합니다.

이유: CLK 에지 직후 L1이 불투명 모드로 전환되는 데 약간의 시간이 걸립니다. 그 전에 D가 바뀌면 L1이 혼란을 겪습니다.

```
D   ──[안정]──────────[새값]──
               tsetup│      │thold
                     ↓      ↓
CLK ─────────────────┤에지├──────
```

측정 방법: tclk-q(clock-to-Q 지연)가 기준값의 10% 이상 증가하는 시점으로 정의합니다.

- Setup Time: 데이터를 점점 늦게 도착하게 하면서 tclk-q가 10% 증가하는 시점
- Hold Time: 데이터를 점점 일찍 도착하게 하면서 tclk-q가 10% 증가하는 시점

FF 기반 파이프라인 타이밍 ★★★

```
Q1 ──[조합 논리 CL]──→ D2
↑                        ↑
FF1 (CLK1)            FF2 (CLK2)
```

최대 지연 조건 (Setup Time 위반 방지):

FF1이 Q1을 출력한 후, 조합 논리를 거쳐 FF2의 D2에 도달하고, 다음 클럭 에지 전에 Setup Time이 확보되어야 합니다.

Tc ≥ tpcq + tpd + tsetup

- Tc: 클럭 주기
- tpcq: FF1의 Clock-to-Q 전파 지연
- tpd: 조합 논리의 전파 지연 (설계자가 조절하는 파라미터)

- tsetup: FF2의 Setup Time

설계자가 조절할 수 있는 것은 **tpd** 하나입니다.

tpd ≤ Tc - tpcq - tsetup

이것이 Critical Path 조건입니다.

클럭 스큐(Clock Skew)가 있을 때:

FF1과 FF2에 클럭이 동시에 도달하지 않습니다. 이 차이가 tskew입니다.

Setup Time 최악 상황: FF2 클럭이 FF1보다 먼저 도달 → 사실상 Tc가 줄어드는 효과

tpd ≤ Tc - tpcq - tsetup - tskew

최소 지연 조건 (Hold Time 위반 방지):

FF1이 새 값을 출력한 후, 그 값이 너무 빨리 FF2에 도달해서 FF2가 현재 클럭 에지를 처리 중인데 새 데이터가 들어오면 안 됩니다.

tccq + tcd ≥ thold

- tccq: FF1의 Clock-to-Q 오염 지연(처음 흔들리기 시작하는 최소 시간)
- tcd: 조합 논리의 오염 지연

전파 지연(tpd) vs 오염 지연(tcd): tpd는 출력이 최종 안정값에 도달하는 시간(최악), tcd는 처음으로 흔들리기 시작하는 시간(최선). 항상 tcd ≤ tpd입니다.

클럭 스큐 포함: **tcd + tccq ≥ thold + tskew**

Hold Time 최악 상황: FF2 클럭이 FF1보다 늦게 도달 → 빠른 경로의 새 데이터가 FF2 샘플링 전에 도착

Pulsed Latch 기반 파이프라인

FF 대신 매우 짧은 펄스(폭 Tpw)로 동작하는 래치를 사용합니다.

최대 지연: tpd ≤ Tc - tpcq - tsetup **+ Tpw** (Time Borrowing)

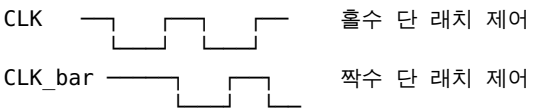
Tpw만큼 조합 논리에 더 많은 시간을 줄 수 있습니다. 래치가 투명한 동안(Tpw) 데이터가 늦게 도착해도 바로 통과하기 때문입니다.

최소 지연: tcd ≥ thold - tccq **+ Tpw** (Hold 조건 강화)

반대로 Hold Time 조건은 더 엄격해집니다. 래치가 열려 있는 시간 동안 이전 데이터가 흘러들어올 수 있기 때문입니다.

Two-Phase Latch 기반 파이프라인

두 개의 상보적 클럭(CLK, CLK_bar)으로 래치를 교번 동작시킵니다.



최대 지연: Tc ≥ tpdq1 + tpd1 + tpdq2 + tpd2

놀랍게도 **Setup Time이 사라진 것처럼 보입니다!**

이유: 래치가 투명한 동안은 데이터가 그냥 통과합니다. 래치가 닫히기 전까지만 도착하면 되므로, 에지 기준의 tsetup이 없습니다.

하지만 Setup Time이 완전히 사라진 게 아닙니다. **Time Borrowing 제약** 형태로 숨어 있습니다.

Time Borrowing: L2가 열려 있는 시간은 Tc/2입니다. 이 동안 Q1이 늦게 도착해서 L2를 통과해도 됩니다. 단, L2가 닫히기 tsetup 전까지는 안정되어야 합니다.

최대 borrowing 시간: tborrow = Tc/2 - tsetup

최소 지연: tccq + tcd ≥ thold (매 Tc/2마다 성립)

Ch7. Dynamic Circuits

이 챕터는 왜 필요한가?

Ch4에서 Logical Effort를 배울 때, PMOS의 낮은 이동도가 NOR 게이트를 느리게 만드는 주요 원인이었습니다. PMOS를 논리 연산에서 아예 제거하고, 충전(프리차지)에만 쓰면 어떨까요? 이것이 다이나믹 회로의 핵심 아이디어입니다.

다이나믹 회로의 기본 동작

CMOS 인버터에서 PMOS PUN을 없애고, 클럭으로 제어되는 단일 PMOS 프리차지 트랜지스터만 남깁니다.

```
        VDD
         |
[PMOS, CLK=0 → ON]  ← 프리차지 트랜지스터
         |
         Y (출력 노드)
         |
[NMOS PDN]  ← 논리 연산 (NMOS만으로 구성)
         |
        GND
```

CLK=0 (프리차지 단계): PMOS가 켜져서 Y를 VDD로 충전합니다. 논리 연산 없음.

CLK=1 (평가 단계): PMOS가 꺼집니다. PDN이 동작합니다. 입력에 따라 Y가 GND로 방전될 수도(출력=0), 그대로 VDD를 유지할 수도(출력=1) 있습니다.

장점: - PUN이 없으므로 트랜지스터 수 감소 - LE가 감소 (PMOS 없이 NMOS만으로 논리 구현) - NMOS만의 방전이라 더 빠름

Unfooted vs. Footed

Unfooted 다이나믹:

PDN 아래에 CLK 제어 NMOS가 없습니다.

- **장점**: PDN의 LE가 작아 빠릅니다

- **단점**: 프리차지 중(CLK=0) 입력이 1이면 PDN이 켜져 프리차지 PMOS와 충돌(Contention) 발생 → 프리차지 중 입력이 반드시 0이어야 합니다

Footed 다이나믹:

PDN 아래에 CLK=1일 때 켜지는 "풋(Foot)" NMOS가 추가됩니다.

- **장점**: CLK=0이면 풋 트랜지스터가 꺼져서 입력이 1이어도 충돌 없음
- **단점**: 직렬 NMOS 하나 더 → PDN 저항 증가 → LE 나빠짐

단조성(Monotonicity): 다이나믹 회로에서 평가 단계 중 출력은 1→0(방전)만 가능합니다. 0→1(충전)은 다음 프리차지 단계에서만 가능합니다. 이것이 단조성입니다.

단조성 문제 (Monotonicity Problem)

두 개의 다이나믹 게이트를 직렬로 연결하면 심각한 문제가 생깁니다.

```
1단 출력 X → 2단 PDN 입력
```

평가가 시작되는 순간(CLK: 0→1), 1단의 출력 X는 **프리차지로 인해 아직 1**입니다. X가 실제로는 0이 되어야 하지만(예: A=1이어서), 처음에는 프리차지 전압(1)을 유지합니다.

2단의 PDN은 X=1을 보고 Y를 방전시켜버립니다. 그 다음 X가 0으로 바뀌지만 이미 Y는 잘못 방전된 상태입니다. 오평가(False Evaluation)가 발생합니다.

도미노 로직 (Domino Logic) ★★★

단조성 문제의 해결책: **다이나믹 게이트 + 정적 인버터**를 결합합니다.

```
      VDD
       |
   [PMOS 프리차지]
       |
       X ──→ [정적 인버터] ──→ Y (출력)
       |                  ↑
   [NMOS PDN]          Keeper(약한PMOS)
       |
      GND
```

프리차지 단계: X=VDD → 인버터 → Y=0

평가 단계: - PDN이 방전하면: X→0 → Y=1 - PDN이 방전 안 하면: X=1 유지 → Y=0

Y는 항상 0에서 시작해서 0→1만 가능합니다. 다음 단의 PDN 입력이 항상 0에서 시작하므로 잘못 평가할 일이 없습니다.

마치 도미노 패가 차례로 쓰러지듯 앞에서 뒤로 순서대로 평가가 진행됩니다. 그래서 "도미노"입니다.

중요한 특성: 도미노 로직은 **비반전(Non-inverting)** 출력만 가능합니다. 다이나믹 게이트가 반전이지만 인버터가 다시 반전시켜 결과적으로 비반전입니다. XOR 같은 연산을 직접 구현하기 어렵습니다.

Keeper — 누설 전류 문제 해결

평가 결과 X=1을 유지해야 하는데, OFF 상태 NMOS의 서브스레숄드 누설 전류가 X를 서서히 방전시킵니다. 이것이 잘못된 평가로 이어질 수 있습니다.

해결책: 출력 X에 **약한 PMOS Keeper**를 추가합니다.

```
인버터 출력 Y=0이면 → Keeper PMOS가 ON → X를 VDD로 당겨줌
```

Keeper 크기 설계의 딜레마:

- **너무 강하면**: 실제 평가 때 PDN이 X를 방전시키려 해도 Keeper가 방해 → 오동작
- **너무 약하면**: 누설 전류를 이기지 못함 → X가 방전 → 오동작

Keeper는 **누설 전류는 이길 수 있지만, 평가 시 PDN 전류는 이길 수 없는** 절묘한 크기여야 합니다.

전하 공유 문제 (Charge Sharing)

PDN이 여러 NMOS를 직렬로 연결할 때, 내부 노드에도 커패시턴스가 있습니다.

```
      VDD
       |
   [PMOS]
       |
       X2 (C2 = VDD로 프리차지)
       |
   [NMOS A=1]
       |
       X1 (C1 = ??? 프리차지 안 됨)
       |
   [NMOS B=0]  ← B=0이므로 GND와 차단
       |
      GND
```

평가 시작 → A=1이므로 A 아래 NMOS가 켜짐 → X2(VDD)와 X1(부동)이 연결됨

전하 공유로 X2의 전압이 내려감:

$V(X2) = VDD \times C2/(C1 + C2)$

X2가 인버터 트립 전압 아래로 떨어지면 Y가 오동작합니다.

해결책: X1도 프리차지 단계에서 VDD로 충전해 둡니다. X1도 VDD이면 X2와 X1이 연결되어도 전압 변화가 없습니다.

도미노 파이프라인

도미노 회로를 파이프라인에 쓸 때, 모든 게이트가 같은 클럭을 쓰면 클럭의 절반(프리차지 구간)이 낭비됩니다.

해결책: CL1과 CL2에 반대 위상의 클럭을 공급합니다.

```
CLK    ──┐      ┌──────────
         └──────┘

CL1    [프리차지][──평가──][프리차지]
CL2    [──평가──][프리차지][──평가──]
```

CL1이 평가하는 동안 CL2는 프리차지, CL2가 평가하는 동안 CL1은 프리차지.

전체 클럭 주기를 유효하게 사용합니다.

Footed/Unfooted 사용 규칙

FF 직후 첫 번째 다이나믹 단은 반드시 Footed여야 합니다.

이유: FF의 출력 Q는 CLK=0(프리차지 시작) 이후에도 한동안 1을 유지할 수 있습니다. Q=1인 상태에서 Unfooted 게이트가 프리차지를 시작하면 PDN이 켜진 채로 프리차지 PMOS와 충돌이 생깁니다.

Footed 게이트는 CLK=0이면 풋 트랜지스터가 꺼져서 Q=1이어도 충돌이 없습니다.

이후 단은 Unfooted를 써도 됩니다.

도미노 로직의 출력은 프리차지 후 항상 0에서 시작합니다. Unfooted 게이트의 입력이 프리차지 때 항상 0임이 보장됩니다.

Unfooted 프리차지의 물결(Ripple) 문제

CLK: 1→0 (프리차지 시작) 시점에 앞 단의 출력이 아직 1인 경우가 있습니다. 프리차지 신호가 앞에서 뒤로 물결처럼 전파되면서 각 게이트가 순간적으로 충돌 전류를 겪습니다. 이 후속 클럭을 약간 지연시켜 처리하기도 합니다.

차동 도미노 (Differential/Dual-Rail Domino)

도미노의 비반전 제약을 해결합니다. 함수 F와 그 보수 F'를 동시에 계산하는 두 다이나믹 게이트를 나란히 놓습니다.

- Out = A·B와 Out_bar = ~(A·B)를 동시 출력
- 다음 단에서 반전이 필요하면 Out_bar를 씁니다
- XOR/XNOR 같은 복잡한 함수도 구현 가능
- 단점: 트랜지스터 수가 거의 2배

np-CMOS

NMOS 기반 다이나믹 게이트와 PMOS 기반 다이나믹 게이트를 교번 연결합니다. 이론적으로는 인버터 없이 반전 출력도 가능하지만, 노이즈에 극도로 취약하여 실제로는 거의 사용하지 않습니다.

다이나믹 회로의 현재 위치

전력 소비 증가: 매 사이클마다 프리차지 트랜지스터가 스위칭합니다. A=1이 계속 입력되는 도미노 인버터는 정적 CMOS 인버터보다 스위칭 활동이 높습니다.

노이즈 내성 낮음: 공정이 미세화될수록 누설 전류와 커플링 노이즈가 심해져 다이나믹 회로 설계가 더 어려워집니다.

현재 사용처: 고성능 프로세서의 데이터패스에서 한때 매우 인기 있었으나, 지금은 정적 CMOS가 대부분 차지합니다. 여전히 **메모리 배열의 읽기/쓰기 경로**에 사용됩니다.

도미노 회로가 메모리(계층적 비트라인)에 적합한 이유

와이드 OR 구조와 도미노의 궁합

SRAM 배열에서 읽기 동작을 생각해봅시다. 한 컬럼(비트라인)에 수백~수천 개의 셀이 연결되어 있습니다. 읽기 시 선택된 셀 하나만 BL을 방전시킵니다. 이것은 본질적으로 "수백 개 중 하나라도 방전하면 출력이 바뀌는" 구조입니다. 즉 **와이드 OR(Wide OR)**입니다.

```
비트라인(BL)  ← 프리차지 후 플로팅
    │
    ├─[셀0] — WL0    (Q=0이면 방전 경로 생김)
    ├─[셀1] — WL1
    ├─[셀2] — WL2    ← 이 중 선택된 하나만 WL=1
    │  ...
    └─[셀N] — WLN

결과: 선택 셀의 Q=0이면 BL 방전 (출력=1)
      선택 셀의 Q=1이면 BL 유지  (출력=0)
```

이 구조가 도미노 로직의 동작과 **완벽하게 일치**합니다:

- 프리차지: BL=VDD (도미노의 출력 노드=VDD)
- 평가: 선택 셀의 PDN이 조건부 방전 (도미노의 조건부 방전)
- 결과: 0→1 전환만 가능 (도미노의 단조성)

계층적 비트라인(Hierarchical Bit Lines)에서 도미노 사용

비트라인이 길면 Cbl이 커져 ΔV가 작아집니다. 해결책으로 로컬 BL과 글로벌 BL을 나눕니다.

```
글로벌 BL (긴 배선, 큰 C)
    │
    ├─[선택 회로 A]— 로컬 BL0 —[셀][셀][셀]...[셀] (16~64개)
    │
    ├─[선택 회로 B]— 로컬 BL1 —[셀][셀][셀]...[셀]
    │
    └─[선택 회로 C]— 로컬 BL2 —[셀][셀][셀]...[셀]
```

로컬 BL에는 소수의 셀만 연결되어 Cbl_local이 작아집니다. ΔV가 커지고 빠르게 읽을 수 있습니다.

여기서 도미노 회로가 선택 회로 역할을 담당합니다.

```
[도미노 선택 회로 동작]

프리차지 단계 (CLK=0):
  글로벌 BL → VDD 충전 (도미노 출력 노드)
  로컬 BL들도 VDD로 프리차지

평가 단계 (CLK=1):
  선택된 로컬 BL(WL=1로 셀 읽기 중):
    해당 셀 Q=0 → 로컬 BL 방전 → 도미노 PDN ON
    → 글로벌 BL 방전 → 센스 앰프가 감지

  비선택 로컬 BL:
    WL=0이므로 셀 접근 안 됨 → 글로벌 BL에 영향 없음
```

도미노가 계층적 비트라인에 적합한 구체적 이유:

1. **단조성과 일치**: 비트라인은 프리차지(HIGH) 후 방전(LOW)만 합니다. 도미노 로직도 출력이 0→1만 가능(프리차지 후 방전). 동작 방향이 완벽히 일치합니다.

2. **와이드 OR 구현**: 로컬 BL 여러 개 중 하나라도 방전되면 글로벌 BL을 방전시키는 구조가 도미노의 PDN 병렬 구조(OR)와 동일합니다.

3. **속도**: 도미노의 PDN은 NMOS만이라 정적 CMOS PUN 없이 빠른 방전이 가능합니다. BL 방전이 빨라야 읽기 속도가 빠릅니다.

4. **면적 효율**: 정적 CMOS로 선택 회로를 구현하면 PMOS PUN이 추가되어 면적이 증가합니다. 도미노는 프리차지 PMOS 하나로 충분합니다.

```
[도미노 기반 계층적 비트라인 회로도]

         VDD
          |
     [PMOS, CLK=0 → ON]  ← 글로벌 BL 프리차지
          |
     글로벌 BL ──────────────────────────────
          |
    ┌─────┴──────┬──────────────┐
    |            |              |
[NMOS A0]  [NMOS A1]    [NMOS A2]  ← 로컬 BL 선택 (addr)
    |            |              |
로컬BL0    로컬BL1       로컬BL2
    |            |              |
  [셀들]      [셀들]         [셀들]
   WL0~WLk   WL0~WLk       WL0~WLk

동작:
  CLK=0: 글로벌 BL 프리차지 → VDD
  CLK=1: addr로 선택된 로컬 BL의 셀이 방전되면
         해당 NMOS를 통해 글로벌 BL도 방전
         → 센스 앰프가 글로벌 BL 변화 감지
```

정리: 도미노 회로는 메모리의 "프리차지 → 조건부 방전 → 센스"라는 읽기 사이클과 구조적으로 동일하기 때문에, 추가 로직 없이 자연스럽게 메모리 읽기 경로에 통합됩니다. 이것이 다이나믹 회로가 데이터패스에서는 밀려났지만 메모리에서는 여전히 사용되는 핵심 이유입니다.

Ch8. Memory Cell Operations

이 챕터는 왜 필요한가?

처리할 데이터를 어딘가에 저장해야 합니다. 메모리가 바로 그 역할입니다. 이 챕터는 DRAM(메인 메모리)과 Flash(비휘발성 스토리지)의 회로 수준 동작을 다룹니다.

SW 개발자 관점에서: 여러분이 malloc()을 호출하면 DRAM에 공간이 할당되고, 파일을 저장하면 Flash에 기록됩니다. 이 소자들이 어떻게 동작하는지 알면 메모리 성능을 더 잘 이해할 수 있습니다.

DRAM — 트랜지스터 1개 + 커패시터 1개

DRAM(Dynamic Random Access Memory)은 **1T-1C** 구조입니다. 1970년 IBM의 Bob Dennard가 발명했습니다.

```
Word Line (WL) ──┤ NMOS ├──────────── Bit Line (BL)
                        |
                [커패시터 C_cell]  ← 데이터 저장
                        |
                       GND
```

Word Line(WL): 특정 행의 모든 셀을 선택하는 신호

Bit Line(BL): 특정 열의 데이터 입출력 선

C_cell: 실제 데이터를 저장. 전하가 있으면 "1", 없으면 "0"

"Dynamic"의 의미: 커패시터에 저장된 전하가 누설로 인해 시간이 지나면 자연적으로 사라집니다. 데이터가 손실됩니다. 그래서 **64ms마다 리프레시(Refresh)**를 해야 합니다. 리프레시 중에는 메모리를 정상적으로 사용할 수 없어 성능에 영향을 줍니다.

DRAM 쓰기 동작

"1" 쓰기: BL = VDD(HIGH), WL = 1 → NMOS ON → C_cell이 VDD로 충전

"0" 쓰기: BL = GND(LOW), WL = 1 → NMOS ON → C_cell이 GND로 방전

DRAM 읽기 동작 ★★★

읽기는 훨씬 복잡합니다.

① BL을 VDD/2로 프리차지 후 플로팅(Floating) 상태로 만들기

② WL = 1로 켜기 → C_cell과 BL이 연결됨

③ 전하 공유로 BL 전압이 미세하게 변함

전하 보존 법칙에 의해:

"1"이 저장된 경우(C_cell = VDD): ΔVbl = +Ccell × VDD/2 / (Cbl + Ccell)

"0"이 저장된 경우(C_cell = 0): ΔVbl = -Ccell × VDD/2 / (Cbl + Ccell)

문제: Cbl(비트라인 커패시턴스)은 그 열에 연결된 수백~수천 개 셀의 커패시턴스 합으로 Ccell보다 훨씬 큽니다.

예를 들어 Ccell = 30fF, Cbl = 600fF이면:

ΔVbl = 30 × VDD/2 / 630 ≈ 0.048 × VDD/2

VDD = 1.8V라면 ΔVbl ≈ 43mV밖에 안 됩니다. 이 작은 신호를 0과 1로 판별하는 회로가 **센스 앰프(Sense Amplifier)**입니다.

파괴적 읽기(Destructive Read): 읽기 과정에서 C_cell의 전하가 BL과 공유되어 손상됩니다. 읽기 후에 반드시 **복원(Restore)** 동작이 필요합니다. 센스 앰프가 증폭된 신호를 다시 셀에 써줍니다.

eDRAM — 온칩 DRAM

일반 DRAM은 별도 칩에 있어 CPU와의 통신이 느립니다. eDRAM은 CPU와 같은 칩에 DRAM을 올린 것입니다.

3T eDRAM Gain Cell (Intel):

트랜지스터 3개로 구성됩니다. 쓰기 포트(WWL/WBL)와 읽기 포트(RBL)가 분리되어 있습니다.

왜 PMOS를 사용하는가? PMOS는 NMOS보다 게이트 누설 전류가 작아서 저장된 전하가 오래 유지됩니다.

읽기 동작:

- RBL을 GND로 프리방전
- 저장 노드 “0” → 읽기 트랜지스터 약하게 ON → RBL이 약간 올라감 (≈200mV)
- 저장 노드 “1” → 읽기 트랜지스터 OFF → RBL이 GND 유지

비파괴 읽기(Non-destructive Read): 읽기 경로가 저장 노드와 완전히 분리되어 있어 읽어도 저장 값이 손상되지 않습니다. 복원이 필요 없어 더 빠릅니다.

데이터 유지 특성:

- “1” 유지 시간 > “0” 유지 시간
- 이유: 저장 노드의 주변 바이어스 조건이 “1” 상태에서 자기 역방향 바이어스를 만들어 누설을 줄임
- 시간이 지날수록 “0”의 전압이 높아지며 읽기 속도가 느려짐

IBM eDRAM (Power7 L3 캐시): 1T-1C 구조. 딥 트렌치(Deep Trench) 커패시터 공정 사용.

Intel eDRAM (Haswell Iris): 1T-1C 구조. MIM(Metal-Insulator-Metal) 커패시터를 기존 인터커넥트(M2~M4) 사이에 형성. CPU와 같은 다이가 아닌 같은 패키지 내 별도 칩.

Flash Memory — 전원 꺼져도 데이터 유지

DRAM은 전원이 꺼지면 데이터가 사라집니다. Flash는 전원 없이도 데이터를 유지하는 **비휘발성(Non-volatile)** 메모리입니다. 도시바의 마스오카 후지오가 발명했습니다.

Flash 셀 구조 — 플로팅 게이트

일반 MOSFET과 달리 게이트가 두 개 있습니다.

```
컨트롤 게이트 (Control Gate) ← 외부에서 전압 인가
       |
  [인터폴리 유전체]
  [플로팅 게이트]  ← 완전히 절연. 전하 영구 저장
  [터널링 산화막] (~10nm)
  N+──────────────N+
  S     P형 기판   D
```

컨트롤 게이트(Control Gate): 외부에서 접근 가능. 프로그래밍/읽기 전압을 인가합니다.

플로팅 게이트(Floating Gate): 절연층에 완전히 둘러싸인 도체. 외부 단자가 없습니다. 여기에 전자를 가두거나 빼내어 데이터를 저장합니다.

데이터 표현 원리:

플로팅 게이트에 전자가 없으면 → Vth 낮음 → 낮은 읽기 전압에서 전류 흐름 → **"1"**

플로팅 게이트에 전자 있으면 → 음전하가 채널을 차폐 → Vth 높아짐 → 전류 안 흐름 → **"0"**

Flash 프로그래밍(쓰기 "0") — 핫 전자 주입

컨트롤 게이트에 매우 높은 전압(≈12V), 드레인에 높은 전압(≈7V) 인가.

채널에서 드레인 방향으로 흐르는 전자들이 강한 전기장으로 매우 높은 에너지("뜨거운" 전자)를 얻습니다. 이 에너지가 터널링 산화막의 에너지 장벽을 넘어 플로팅 게이트로 주입됩니다.

고전압이 필요하고, 반복될수록 터널링 산화막이 열화됩니다. 이것이 Flash 내구성 제한의 원인입니다. 통상 10,000~100,000회 쓰기 가능.

Flash 소거 ("1") — FN 터널링

소스에 매우 높은 전압(≈20V), 컨트롤 게이트에 낮은 전압(또는 0V) 인가.

플로팅 게이트의 전자들이 강한 전기장에 의해 **양자역학적 터널링**으로 얇은 산화막을 통과해 기판으로 탈출합니다.

블록 단위 소거: FN 터널링은 모든 전자를 동시에 이동시킵니다. 따라서 소거는 셀 단위가 아닌 블록 단위로 합니다. 이것이 Flash에서 임의 쓰기(Random Write)가 느린 이유입니다. 쓰기 전에 해당 블록 전체를 소거해야 하기 때문입니다.

NAND vs NOR Flash ★★★

NAND Flash: 직렬 연결

```
BL
|
[DSL] ← Drain Select
|
[셀3] —WL3
|
[셀2] —WL2  ← 선택 셀: Vread=0V
|              비선택 셀: Vpass=4~5V
[셀1] —WL1
|
[SSL] ← Source Select
|
GND
```

셀들이 직렬로 연결되어 스트링(String)을 이룹니다.

- 고밀도: 셀들이 직렬이라 각 셀의 접촉 면적을 줄일 수 있음
- 페이지 단위 읽기: 한 번에 한 행 전체를 읽음
- 임의 접근 어려움
- 용도: SSD, USB, 스마트폰 저장소

NAND 읽기 원리:

- 읽고 싶은 셀(WL[1]): Vread = 0V 인가
- 나머지 셀들: Vpass = 4~5V 인가 (강제로 ON)

Vpass는 어떤 Vth를 가진 셀도 강제로 켭니다. 오직 선택 셀만 Vread에 의해 ON/OFF가 결정됩니다.

- 선택 셀 "1": Vth 낮음 → Vread(0V) > Vth → ON → 전류 흐름 → BL 방전
- 선택 셀 "0": Vth 높음 → Vread(0V) < Vth → OFF → 전류 없음 → BL 유지

NOR Flash: 병렬 연결

각 셀이 비트라인에 병렬로 연결됩니다. NOR 게이트 구조와 같습니다.

- 임의 접근 가능 → XIP(Execute In Place) 가능: Flash에서 코드를 직접 실행
- 셀 크기 큼 (병렬 연결로 접촉 면적 필요)
- 용도: 펌웨어, 부팅 코드 저장

Ch9. SRAM Design

이 챕터는 왜 필요한가?

CPU 바로 옆에 붙어있는 캐시(Cache) 메모리는 DRAM처럼 리프레시가 필요 없고, 매우 빠르게 동작해야 합니다. 이것이 SRAM(Static RAM)입니다.

소프트웨어 관점에서: L1/L2/L3 캐시가 SRAM입니다. 이 챕터는 CPU 성능에 직접적인 영향을 미치는 SRAM이 어떻게 동작하는지, 왜 설계가 어려운지를 다룹니다.

6T SRAM 셀 구조

두 개의 인버터를 교차 연결(Back-to-Back)한 구조입니다.

```
BL          BLb
|            |
[XL]        [XR]   ← 접근 트랜지스터 (WL로 제어)
|            |
Q ──┬──[PL]──→ Qb ──┬──[PR]──→ Q
    |   |           |   |
   [NL] |          [NR] |
    |   VDD         |   VDD
   GND              GND
```

6개 트랜지스터:

- NL, PL: 왼쪽 인버터 (NMOS + PMOS)
- NR, PR: 오른쪽 인버터 (NMOS + PMOS)
- XL, XR: 접근 트랜지스터 (Word Line으로 제어)

"Static"의 의미: 두 인버터의 피드백 루프가 데이터를 자동 유지합니다. Q=1이면 Qb=0, Qb=0이면 Q=1. 전원이 있는 한 리프레시 없이 영구 유지됩니다.

쌍안정(Bi-stable): 두 가지 안정 상태 (Q=1/Qb=0, Q=0/Qb=1)

SRAM Hold 동작

WL = 0 → XL, XR 꺼짐 → 셀이 비트라인과 완전히 분리됨 → 피드백 루프가 현재 값 유지

SRAM 읽기 동작 ★★★

```
① BL = BLb = VDD로 프리차지
② WL = VDD → XL, XR 켜짐
③ Q=0인 쪽의 BL이 방전 시작 → BL과 BLb 사이 전압 차이 발생
④ 센스 앰프가 이 차이를 증폭
```

읽기 파괴(Read Disturb) 위험:

WL이 켜지면 XR을 통해 BL(VDD)에서 Q 노드로 전류가 흘러 Q 전압이 올라갑니다. 만약 Q 전압이 충분히 올라가서 오른쪽 인버터(PR+NR)의 트립 전압을 넘으면, 데이터가 뒤집힙니다!

안전 조건: **NR이 XR보다 강해야 합니다** (NR의 ON 저항 < XR의 ON 저항). NR이 Q 노드를 GND로 강하게 잡아줘야 XR의 충전 전류를 이길 수 있습니다.

SRAM 쓰기 동작 ★★★

"0"을 Q에 쓰는 경우 (현재 Q=1):

```
BL = GND (쓰고 싶은 값)
BLb = VDD
WL = VDD → XL, XR 켜짐
```

XL을 통해 GND가 Q(현재 VDD) 노드와 연결됩니다. **전투(Fight)**가 벌어집니다:

- XL(접근 트랜지스터): Q를 GND 쪽으로 끌어내리려 함
- PL(왼쪽 인버터의 PMOS): Q=1을 유지하려고 VDD로 올리려 함

쓰기 성공 조건: XL이 PL보다 강해야 합니다. XL이 PL을 이겨야 Q 노드를 낮출 수 있습니다.

Q가 낮아지면 → 오른쪽 인버터가 Qb를 올림 → 피드백이 걸려 Q가 더 낮아짐 → 새 데이터 "0" 안착.

Art of Balancing: NR:XR:PR ≈ 2:1:1 ★★★

세 가지 트랜지스터 크기가 서로 상충하는 요구를 가집니다.

Large NR (드라이브 NMOS 크게):

- 장점: NR이 클수록 읽기 전류(Iread) 증가 → 빠른 읽기
- 부작용: NR이 너무 크면 왼쪽 인버터(PL-NL)의 트립 전압이 낮아짐. 읽기 때 XR을 통해 Q 노드가 약간 올라가도 인버터가 반응 → 셀 불안정

Large XR (접근 트랜지스터 크게):

- 장점: BL과 셀 사이 전류 증가 → 빠른 읽기/쓰기
- 부작용: XR이 너무 크면 읽기 때 BLb(VDD)에서 Q 노드로 강한 전류가 흘러 Q 전압이 너무 높이 올라감 → 데이터 뒤집힘

Large PR (로드 PMOS 크게):

- 장점: 피드백이 강해져 Q=0 상태를 더 강하게 유지 → SNM 향상
- 부작용: PR이 너무 크면 쓰기 때 XL이 Q 노드를 낮추려 해도 PR이 강하게 VDD로 당겨서 쓰기 실패

세 트랜지스터가 서로 시소 관계입니다. 균형점이 **NR:XR:PR ≈ 2:1:1**입니다. NR을 가장 크게 하고 XR과 PR을 같게 합니다.

SRAM 어레이 구조

M×N 어레이: M개 행(WL), N개 열(BL/BLb)

문제: 어레이가 높고 좁은(Tall) 형태가 됩니다. 비트라인이 길어질수록 **비트라인 커패시턴스(Cbl)**가 커집니다. 읽기 시 ΔV가 더 작아지고 속도도 느려집니다.

센스 앰프 (Sense Amplifier) ★★★

BL과 BLb 사이의 작은 전압 차이(수십~수백 mV)를 레일-투-레일(0V 또는 VDD)로 증폭합니다. SRAM 읽기 속도와 정확도를 결정하는 핵심 회로입니다.

차동 쌍 센스 앰프 (Differential Pair Sense Amplifier)

```
[회로도]
        VDD
         |
    ┌───[P1]───┬───[P2]───┐
    │          |          |
  sense_b      |        sense
    |        [N3]         |
    └───[N1]───┴───[N2]───┘
         |              |
        BL            BLb
         |              |
        GND

P1, P2: PMOS 로드
N1, N2: NMOS 입력 쌍 (BL, BLb에 연결)
N3: 바이어스 전류원 NMOS
```

동작 원리: BL < BLb이면 N1보다 N2에 전류가 더 많이 흐릅니다. N1 쪽에 전류가 적으므로 P1이 sense_b를 VDD 쪽으로 올리고, N2 쪽에 전류가 많으므로 sense가 GND 쪽으로 내려갑니다. 이 차이가 증폭됩니다.

장점: 클럭 없이 동작(비동기). BL 전압 차이가 생기는 즉시 자동으로 증폭을 시작합니다.

단점: N3 바이어스 전류원이 항상 전류를 흘려보냅니다. 정적 전력 소비가 큽니다. BL 전압 차이가 작으면 정확한 판별이 어렵습니다. 도미노 회로처럼 레일-투-레일 풀 스윙이 필요한 경우 성능이 부족합니다.

클럭드 센스 앰프 Type 1 — 격리 트랜지스터 포함 (주류)

```
[회로도]
      BL        BLb
       |          |
  [Miso_L]    [Miso_R]  ← 격리 트랜지스터 (sense_clk으로 제어)
```

```
       |          |
      bit       bit_b   ← 내부 노드
       |          |
  ┌──[N1]────[N2]──┐  ← NMOS 입력 쌍
  │    |        |   |
  │   [N3: sense_clk]   ← 풀다운 NMOS
  │         |
  │        GND
  │
  ├──[P1]──┬──[P2]──┤  ← PMOS 재생 쌍 (교차 연결)
  │         |         |
sense_b   공통      sense
  |                   |
 VDD               VDD
```

실제 회로를 더 명확하게 표현하면:

```
           VDD    VDD
            |      |
           [P1]   [P2]
            |      |
sense_b ───┤      ├─── sense   ← 출력
            |      |
     BL ──[N1]  [N2]── BLb    ← 입력
            |      |
           [격리 트랜지스터들: sense_clk]
            |      |
           [N3: sense_clk]     ← 공통 풀다운
            |
           GND

교차 연결: P1의 게이트 = sense, P2의 게이트 = sense_b
           → 양성 피드백(Positive Feedback) = 재생(Regenerative) 피드백
```

동작 원리 (4단계):

① 프리차지 (sense_clk=0)

sense_clk=0이면 N3(공통 풀다운)이 꺼집니다. sense와 sense_b 내부 노드가 VDD 쪽으로 프리차지됩니다. 격리 트랜지스터(Miso)도 꺼져 BL과 분리됩니다.

② 격리 해제 (sense_clk=0→1 직전)

Miso 트랜지스터가 켜지면서 BL과 BLb의 전압 차이가 내부 노드(bit, bit_b)에 전달됩니다. 이 시점에 아직 N3는 꺼져있어 전류는 흐르지 않습니다. BL의 작은 ΔV가 내부 노드에 그대로 반영됩니다.

③ 재생 증폭 (sense_clk=1)

N3가 켜지면서 차동 전류가 흐르기 시작합니다. BL < BLb이면:

- N1 전류 < N2 전류
- sense_b 전압이 sense보다 높아짐
- P1(게이트=sense)이 더 강하게 켜지면서 sense_b를 더 올림
- P2(게이트=sense_b)가 더 강하게 꺼지면서 sense가 더 내려감
- **양성 피드백 루프**가 작동하여 차이가 기하급수적으로 증폭

수 나노초 안에 sense_b=VDD, sense=0 (또는 반대)으로 수렴합니다.

④ 격리 유지

재생이 완료된 후에도 Miso가 BL을 차단하여, 방전된 BL이 회로에 영향을 주지 않습니다.

격리 트랜지스터(Isolation Transistor)의 핵심 역할:

```
격리 없이:
BL(큰 C) ──[N1]── sense
                  ↑
              재생 피드백이
              BL 충방전에 에너지를 낭비
              → 증폭 느려짐

격리 있이:
BL(큰 C) ──[Miso]──[N1]── sense
                  ↑ 차단!
           센싱 시작 후 BL 분리
           → 작은 커패시턴스만 재생
           → 빠른 증폭
```

BL의 커패시턴스(수백 fF~수 pF)가 재생 루프에 붙어 있으면 증폭이 느려집니다. 격리 트랜지스터로 분리하면 재생 루프의 커패시턴스가 훨씬 작아져(수 fF) 매우 빠르게 수렴합니다.

장점: 정적 전력 0 (sense_clk=0이면 전류 없음). 빠른 증폭(재생 피드백). 레일-투-레일 출력.

단점: sense_clk 타이밍이 매우 중요합니다.

- 너무 일찍: BL ΔV가 충분히 발달 안 함 → 오독(False Read) 위험
- 너무 늦게: 불필요한 대기 → 속도 저하
- 공정 변동으로 타이밍 예측이 어려움 → 연구 과제

클럭드 센스 앰프 Type 2 — 격리 트랜지스터 없는 방식

```
[회로도]
      BL        BLb
       |          |
      [N1]       [N2]   ← NMOS 입력 (BL에 직접 연결)
       |          |
      [N3]       [N4]   ← 두 번째 NMOS 쌍
       |          |
      [N5: clk]         ← 공통 풀다운
       |
      GND

sense ──────────────────→ 출력
sense_b ────────────────→ 출력

(PMOS 재생 피드백은 sense, sense_b 사이에 교차 연결)
```

동작 원리: BL이 센스 앰프 출력과 구조적으로 분리되어 있습니다. 3개의 직렬 NMOS 풀다운으로 구성됩니다. BL 전압 차이가 clk이 켜진 후에도 계속 내부 노드로 업데이트됩니다.

Type 1 vs Type 2 비교:

항목	Type 1 (격리 있음)	Type 2 (격리 없음)
격리 트랜지스터	있음	없음
BL 업데이트	격리 후 차단	계속 업데이트
증폭 속도	빠름 (작은 C)	보통

설계 복잡도	높음	낮음
사용처	고성능 SRAM	일반 SRAM

비트라인 프리컨디셔닝

프리차지: BL=BLb=VDD. 읽기 전 비트라인을 미리 충전.

이퀄라이저: BL=BLb로 맞춤. 이전 읽기에서 남은 전압 차이를 제거.

```
이퀄라이저:
BL ┤eq NMOS├ BLb   (eq 신호로 제어)
```

워드라인 디코더 (Wordline Decoder)

N비트 주소로 2^N개의 WL 중 하나를 선택합니다. SRAM에서 디코더는 셀 어레이의 행 높이(Pitch)에 맞춰 배치해야 하므로 면적과 속도 모두 중요합니다.

방법 1: 직접 디코딩 (Direct Decoding)

개념: 각 WL마다 N입력 AND 게이트 하나를 직접 연결합니다.

CMOS에서 AND = NAND + INV이므로, 실제로는 NAND 게이트를 사용합니다.

```
[4:16 디코더 — 직접 디코딩 회로도]

주소 입력: A3, A2, A1, A0 (및 각각의 반전: A3b, A2b, A1b, A0b)

WL0  = A3b · A2b · A1b · A0b
WL1  = A3b · A2b · A1b · A0
WL2  = A3b · A2b · A1  · A0b
...
WL15 = A3  · A2  · A1  · A0

구현:
A3 ──┐
A2 ──┤
A1 ──┤ [4-input NAND] ──[INV]── WL15
A0 ──┘

A3b──┐
A2 ──┤
A1 ──┤ [4-input NAND] ──[INV]── WL7
A0 ──┘

... (총 16개의 4-input NAND + INV 조합)
```

NAND 게이트 수 계산 (4:16 디코더):

- 4입력 NAND: 16개 (각 WL당 1개)
- INV: 16개 (NAND 출력 반전)
- 인버터 버퍼(주소 반전용): 4개 (A0~A3 각각 반전 생성)

합계: **4-input NAND 16개 + INV 20개 = 총 36개 게이트**

CMOS 트랜지스터 수: - 4-input NAND 1개: PMOS 4개(병렬) + NMOS 4개(직렬) = 8개 - 총: 16 × 8 + 20 × 2 = **168개 트랜지스터**

장점: 간단한 구조, 구현 직관적

단점: 4입력 NAND는 직렬 NMOS 4개 → ON 저항 4배 → 느림. 4입력 NAND의 LE = 5/3 (각 입력). 입력이 많을수록 더 느려짐.

방법 2: 계단식 디코딩 (Cascaded Decoding)

개념: 4입력 NAND 대신 2입력 NAND를 계단식으로 연결합니다.

```
[WL15 = A3·A2·A1·A0 구현 예]

A3 ──┐                    ┌──────────────────── WL15
A2 ──┤[NAND2]──[INV]──   │
                           ├─[NAND2]──[INV]──
A1 ──┐                    │
A0 ──┤[NAND2]──[INV]──   │

실제로는:
A3──┐             ┌────────────────
A2──┤NAND2──INV──┤
                  ├─NAND2──INV── WL15
A1──┐             │
A0──┤NAND2──INV──┘
```

NAND 게이트 수 계산 (4:16 디코더):

- 1단: 2-input NAND 2개 per WL × 16 WL = 32개
- 1단 INV: 32개
- 2단: 2-input NAND 1개 per WL × 16 WL = 16개
- 2단 INV: 16개

합계: **2-input NAND 48개 + INV 48개 = 총 96개 게이트**

트랜지스터 수: - 2-input NAND 1개: PMOS 2개(병렬) + NMOS 2개(직렬) = 4개 - 총: 48 × 4 + 48 × 2 = **288개 트랜지스터** (더 많지만 더 빠름)

장점: 2입력 NAND는 LE = 4/3, 직렬 NMOS 2개만 → 빠름. 지연이 계단식으로 줄어듦.

단점: 게이트 수와 트랜지스터 수가 늘어남.

방법 3: 프리디코딩 (Predecoding) ★★★

개념: 주소를 먼저 그룹으로 나눠 부분 디코딩하고, 최종 단에서 2입력 AND만 씁니다.

```
[4:16 디코더 — 프리디코딩 회로도]

A[1:0] → 1-of-4 프리디코더
A[3:2] → 1-of-4 프리디코더

┌──────────────────────────────────┐
│ 1-of-4 프리디코더 (A1, A0)       │
│                                  │
│ A1b─┐                            │
│ A0b─┤NAND2─INV── a00 (A1=0,A0=0)│
│                                  │
│ A1b─┐                            │
│ A0──┤NAND2─INV── a01 (A1=0,A0=1)│
│                                  │
```

```
│ A1──┐                          │
│ A0b─┤NAND2-INV── a10 (A1=1,A0=0)│
│                                │
│ A1──┐                          │
│ A0──┤NAND2-INV── a11 (A1=1,A0=1)│
└────────────────────────────────┘

┌────────────────────────────────┐
│ 1-of-4 프리디코더 (A3, A2)       │
│                                │
│ A3b─┐                          │
│ A2b─┤NAND2-INV── b00            │
│                                │
│ A3b─┐                          │
│ A2──┤NAND2-INV── b01            │
│                                │
│ A3──┐                          │
│ A2b─┤NAND2-INV── b10            │
│                                │
│ A3──┐                          │
│ A2──┤NAND2-INV── b11            │
└────────────────────────────────┘

최종 단 (16개 WL):
a00─┐
b00─┤NAND2──INV── WL0   (A=0000)

a01─┐
b00─┤NAND2──INV── WL1   (A=0001)

a10─┐
b00─┤NAND2──INV── WL2   (A=0010)

...

a11─┐
b11─┤NAND2──INV── WL15  (A=1111)
```

NAND 게이트 수 계산 (4:16 디코더):

프리디코더 1 (A1, A0): - 2-input NAND 4개 + INV 4개

프리디코더 2 (A3, A2): - 2-input NAND 4개 + INV 4개

최종 단: - 2-input NAND 16개 + INV 16개

합계: **2-input NAND 24개 + INV 24개 = 총 48개 게이트**

트랜지스터 수: 24 × 4 + 24 × 2 = **144개 트랜지스터**

세 방식 NAND 게이트 수 비교표

방식	NAND 게이트 수	INV 수	총 게이트	트랜지스터 수	최대 NAND 입력
직접 디코딩	4-input × 16 = 16	20	36	168	4입력
계단식	2-input × 48 = 48	48	96	288	2입력

프리디코딩	**2-input × 24 = 24**	**24**	**48**	**144**	**2입력**

프리디코딩의 압승: 직접 디코딩 대비 게이트 수 25% 절감 + 2입력 NAND로 속도 개선.

일반화: N비트 주소를 k비트씩 그룹으로 나누면 최적 프리디코딩 가능. 4:16에서 2+2 분할이 최적.

피치 매칭(Pitch Matching): WL 드라이버 회로의 높이를 SRAM 셀 행 높이에 맞춰야 합니다. 맞지 않으면 낭비 면적이 생깁니다.

```
[피치 매칭 필요성]

이상적:                 문제 발생:
┌──WL드라이버──┐      ┌──WL드라이버──┐
│              │      │              │
├──────────────┤      │ (빈 공간)    │
│ 셀 행 높이   │      ├──────────────┤
├──────────────┤      │ 셀 행        │
│              │      ├──────────────┤
└──WL드라이버──┘      └──────────────┘

WL 드라이버 높이 = 셀 행 높이  WL 드라이버 높이 > 셀 행 높이
→ 낭비 없음                    → 빈 공간 낭비
```

계층적 비트라인 (Hierarchical Bit Lines)

문제: 긴 비트라인 = 큰 커패시턴스

SRAM 어레이가 커질수록 비트라인이 길어집니다. 비트라인 커패시턴스 Cbl은 연결된 셀 수에 비례합니다. Cbl이 커지면:

$\Delta V_{bl} = C_{cell} \times V_{DD}/2 / (C_{bl} + C_{cell})$ 가 작아집니다

→ 센스 앰프가 감지해야 할 신호가 더 작아집니다 → 오독 위험 증가, 속도 저하

해결책: 로컬 BL + 글로벌 BL 분리

```
[계층적 비트라인 구조]

글로벌 BL (Global BL) — 길고 큰 커패시턴스
    │
    ├──[SA] ← 글로벌 센스 앰프
    │
    ├─[Mlocal_0]─ 로컬 BL0 ─┬─[셀][셀][셀]...[셀] (16~64개)
    │                       │      ↑
    ├─[Mlocal_1]─ 로컬 BL1  WL0~WLk (소수의 셀만)
    │
    └─[Mlocal_2]─ 로컬 BL2 ─[셀][셀]...[셀]

Mlocal: 로컬 BL을 글로벌 BL에 연결하는 선택 트랜지스터
```

각 로컬 BL에는 16~64개의 셀만 연결됩니다.

Cbl_local << Cbl_global → ΔV_{bl_local}이 커짐 → 읽기 신뢰도 향상

도미노 회로를 선택 회로로 사용

앞서 Ch7에서 설명한 것처럼 도미노 회로가 로컬-글로벌 연결의 선택 회로 역할을 합니다.

```
[동작 흐름 요약]

① CLK=0 (프리차지):
   - 글로벌 BL → VDD
   - 모든 로컬 BL → VDD

② 주소 디코딩:
   - 특정 로컬 BL을 선택 (Mlocal 제어)
   - 해당 행의 WL=1 → 선택 셀 접근

③ CLK=1 (평가):
   - 선택 셀의 Q=0 → 로컬 BL 방전
   - Mlocal 통해 글로벌 BL도 방전
   - Q=1 → BL 유지

④ 글로벌 센스 앰프 동작:
   - 글로벌 BL의 변화 감지 → 데이터 출력
```

장점: 로컬 Cbl이 작아 ΔV 증가. 각 로컬 BL을 별도로 프리차지 가능하여 병렬 읽기도 가능.

단점: 선택 회로(Mlocal) 추가로 면적 증가. 제어 신호 복잡도 증가. 글로벌 BL의 큰 Cbl은 여전히 존재하므로 글로벌 단에서의 ΔV는 작음 → 글로벌 센스 앰프 필요.

컬럼 멀티플렉싱 (Column Multiplexing)

128개 컬럼(BL) 중 16비트만 읽어야 할 때, 선택이 필요합니다.

NMOS 트리 방식: 여러 NMOS를 트리 구조로 연결. 외부 디코더 불필요. 직렬 NMOS로 인해 저항이 크고 느립니다.

컬럼 멀티플렉싱에 NMOS만 쓰는 이유: 전달하는 신호가 항상 방전된(LOW) BL 신호이기 때문에 NMOS만으로도 충분합니다.

외부 디코더 방식: 디코더가 하나의 NMOS 패스 게이트를 선택. 직렬 저항이 하나뿐이라 빠릅니다. 외부 디코더 면적이 추가됩니다.

SRAM 안정성 — 공정 미세화의 도전

전통적인 설계 목표 (1990년대 이전): 고성능, 작은 면적

1990년대 후반부터: 누설 전력 감소

2000년대 중반부터: **큰 Random Vt Fluctuation에 대한 안정적인 셀 동작**

공정이 미세화될수록 트랜지스터가 작아집니다. 작은 트랜지스터는 제조 불균일성에 민감합니다. 같은 설계라도 실제 제조 후 각 트랜지스터의 **문턱 전압(Vth)이 제각각**이 됩니다. 이것이 Random Vt Fluctuation입니다.

SRAM 셀 안의 6개 트랜지스터 중 하나라도 Vth가 크게 틀어지면 셀이 불안정해집니다.

정적 노이즈 마진 (Static Noise Margin, SNM) ★★★

SRAM 셀의 안정성을 정량화합니다.

두 인버터의 VTC를 한 그래프에 그립니다. 하나는 Qb=F(Q), 다른 하나는 Q=G(Qb)로.

```
Qb
1.0│╮  STANDBY
   │ ╲  ╭───╮  ← 눈(Eye)이 넓음
   │  ╰─┤   │
   │    ╰╮  │
   │     ╰──╯  READ ← 눈이 좁아짐
0.0└──────────── Q
   0.0      1.0
```

```
SNM = "눈" 안에 들어가는 최대 정사각형의 한 변 길이
```

SNM이 클수록 노이즈에 강하고 안정적입니다.

Read SNM vs Standby SNM:

- Standby: WL=0, 외부 영향 없음 → 눈이 넓음, SNM 큼
- Read: WL=1, XR을 통해 BL(VDD)이 Q 노드를 교란 → Q=0이어야 할 노드 전압 올라감 → 인버터 VTC 비대칭 → 눈이 좁아짐

Read SNM < Standby SNM. 읽기 동작이 SRAM 안정성의 병목입니다.

Vt 불균일과 SNM:

Vth 불균일이 심하면 두 인버터의 VTC가 대칭이 깨집니다. 극단적 경우 "눈"이 사라지고 셀이 **단안정(Mono-stable)**이 됩니다. 데이터가 한쪽으로만 고정되어 버립니다.

Read SNM 개선 방법들

방법 1: PMOS 강화 (Increased PMOS Strength)

PL, PR을 크게 → 인버터 트립 전압 높아짐 → Q 노드가 올라가도 OK → Read SNM 향상

부작용: PR이 너무 강하면 쓰기 때 XL이 PL을 이기지 못함 → 쓰기 실패

방법 2: 셀 VDD 높이기 (Increased Cell VDD)

셀에 높은 전압(VDDH)을 공급하고, BL은 낮은 전압(VDDL)으로 프리차지합니다.

- 높은 셀 VDD → 트립 전압 높아짐 → Read SNM 향상
- 낮은 BL → 쓰기 때 접근 트랜지스터 게이트 오버드라이브 증가 → 쓰기 용이

Dual VDD Column Muxing (K. Zhang et al., ISSCC 05):

- READ: 선택/비선택 컬럼 모두 VDDH
- WRITE: 선택 컬럼 VDDL, 비선택 컬럼 VDDH

부작용: 이중 전원 회로 복잡

방법 3: 8T SRAM — 읽기/쓰기 경로 분리 ★★★

6T의 근본 문제는 읽기와 쓰기가 같은 경로를 공유한다는 것입니다. 8T는 이것을 분리합니다.

```
쓰기 포트 (기존 6T와 동일):
WWL —[XL]— Q —[XR]— Qb
WBL                  WBLb

읽기 포트 (새로 추가):
Q(저장 노드) ──┤RN1├── [RN2] ── RBL
               (게이트)
RWL ──────────────────────────
```

읽기 시 저장 노드(Q)에 직접 전류가 흐르지 않습니다. Q의 전압이 읽기 NMOS의 게이트를 제어할 뿐입니다. 따라서 읽기가 저장 노드를 교란하지 않습니다.

결과: Read SNM = Standby SNM. 읽기 안정성 문제 해결!

단점: 트랜지스터 6T → 8T로 증가, 셀 면적 약 33% 증가

10T SRAM: 8T의 차동(Dual-ended) 읽기 버전. RBL과 RBLb 두 개의 읽기 비트라인으로 더 빠른 읽기와 더 강한 노이즈 내성을 제공합니다.

이상으로 Ch1~Ch9 수업 정리를 마칩니다. 삼성 DS System Expert Course | 디지털 집적회로 | 서울대 김재준 교수